

TẬP ĐOÀN CÔNG NGHIỆP - VIỄN THÔNG QUÂN ĐỘI

BÁO CÁO MINI-PROJECT

Nghiên cứu Feature Flag và xây dựng hệ thống quản lý phát hành tính năng an toàn

Bùi Đình Bảo

dinhbao.itech@gmail.com - 0975 742 607

Chương trình Viettel Digital Talent 2026

Lĩnh vực: Software Engineer

Mentor:	Vũ Tuấn Đạt
Đơn vị:	Viettel Networks

Lời cảm ơn

Trước hết, em xin gửi lời cảm ơn chân thành đến Ban Tổ chức chương trình Viettel Digital Talent 2026 đã tạo điều kiện để em được tham gia học tập, rèn luyện và phát triển trong một môi trường chuyên nghiệp.

Em xin cảm ơn các anh mentor đã trực tiếp giảng dạy và chia sẻ kiến thức trong các buổi training của chương trình. Mỗi buổi đào tạo với một chủ đề và góc nhìn khác nhau đã mang lại cho em nhiều kiến thức, kinh nghiệm thực tế và những định hướng có giá trị trong quá trình học tập và hoàn thiện bản thân.

Em cũng xin cảm ơn mentor phụ trách Mini Project cùng Ban Tổ chức đã cung cấp định hướng, yêu cầu và những góp ý cần thiết trong quá trình thực hiện. Sự hỗ trợ của các anh đã giúp em xác định rõ mục tiêu, chủ động hơn trong quá trình nghiên cứu và hoàn thành sản phẩm.

Bên cạnh đó, em xin cảm ơn các bạn học viên trong chương trình đã cùng trao đổi, chia sẻ và hỗ trợ nhau trong suốt quá trình tham gia Viettel Digital Talent 2026.

Mặc dù đã dành nhiều thời gian và nỗ lực để hoàn thành báo cáo và sản phẩm, do kinh nghiệm còn hạn chế, nội dung thực hiện khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp từ Ban Tổ chức và các anh/chị mentor để có thể tiếp tục cải thiện và hoàn thiện bản thân.

Em xin chân thành cảm ơn!

TP. Hồ Chí Minh, tháng 7 năm 2026

Học viên thực hiện

Bùi Đình Bảo

Lời mở đầu

Trong giai đoạn đầu của ngành công nghiệp phần mềm, các hệ thống thường được phát triển và phát hành theo những phiên bản lớn, với chu kỳ kéo dài từ vài tuần đến vài tháng. Mỗi lần phát hành thường bao gồm nhiều thay đổi được đóng gói và triển khai cùng nhau. Trong mô hình này, việc đưa mã nguồn lên môi trường production và việc cung cấp tính năng cho người dùng gần như là một hoạt động thống nhất: khi phiên bản mới được triển khai, các tính năng bên trong phiên bản đó cũng đồng thời được phát hành.

Cách tiếp cận trên tương đối phù hợp với những hệ thống có tốc độ thay đổi thấp. Tuy nhiên, khi quy mô phần mềm, số lượng người dùng và tần suất cập nhật ngày càng tăng, việc phát hành nhiều thay đổi cùng lúc làm gia tăng đáng kể rủi ro vận hành. Một lỗi trong tính năng mới có thể ảnh hưởng đến toàn bộ người dùng, trong khi quá trình khắc phục thường yêu cầu rollback phiên bản hoặc thực hiện một lần triển khai mới. Bên cạnh đó, việc duy trì các nhánh phát triển tồn tại trong thời gian dài còn làm phát sinh xung đột mã nguồn, tăng chi phí tích hợp và khiến các thay đổi khó được kiểm chứng sớm.

Sự phát triển của các phương pháp Agile, Continuous Integration, Continuous Delivery và trunk-based development đã thúc đẩy các nhóm phát triển tích hợp những thay đổi nhỏ thường xuyên hơn, đồng thời duy trì mã nguồn ở trạng thái có thể triển khai. Tuy nhiên, không phải mọi phần mã đã được tích hợp đều sẵn sàng để cung cấp ngay cho người dùng. Từ đó hình thành nhu cầu đưa mã nguồn của tính năng mới vào cùng phiên bản với hệ thống hiện tại nhưng vẫn có khả năng che giấu, vô hiệu hóa hoặc chỉ kích hoạt tính năng trong những điều kiện nhất định.

Ở dạng sơ khai, nhu cầu này có thể được giải quyết bằng các biến điều kiện, tham số dòng lệnh, biến môi trường hoặc tệp cấu hình. Những cơ chế bật, tắt hành vi phần mềm như vậy đã tồn tại từ trước khi thuật ngữ Feature Flag được sử dụng phổ biến. Khi hệ thống phát triển lớn

hơn, các cấu hình đơn giản dần được tổ chức thành một cơ chế có cấu trúc, cho phép lựa chọn giữa những luồng xử lý khác nhau tại thời điểm chạy. Cơ chế đó được gọi bằng nhiều tên như Feature Toggle, Feature Flag, Feature Switch hoặc Feature Gate. Pete Hodgson mô tả Feature Toggle là tập hợp các kỹ thuật giúp đội ngũ cung cấp chức năng mới nhanh chóng nhưng vẫn kiểm soát được mức độ an toàn; kỹ thuật này cho phép thay đổi hành vi hệ thống mà không cần sửa đổi và triển khai lại mã nguồn. [1]

Sự phổ biến của Feature Flag gắn liền với một thay đổi quan trọng trong tư duy phát triển phần mềm: deploy không đồng nghĩa với release. Deploy là quá trình đưa một phiên bản mã nguồn vào môi trường vận hành, trong khi release là quyết định cho phép người dùng thực sự tiếp cận một tính năng. Khi hai hoạt động này được tách biệt, đội phát triển có thể triển khai mã nguồn sớm nhưng giữ tính năng ở trạng thái tắt, kiểm thử tính năng trên production với một nhóm nội bộ, phát hành cho một tỷ lệ nhỏ người dùng và chỉ mở rộng phạm vi khi đã có đủ độ tin cậy.

Từ một công tắc bật hoặc tắt đơn giản, Feature Flag dần được sử dụng cho nhiều mục đích khác nhau. Release Flag hỗ trợ tích hợp mã chưa sẵn sàng mà không làm ảnh hưởng đến người dùng; Experiment Flag phục vụ A/B testing và đánh giá tác động của các phương án sản phẩm; Permission Flag giới hạn tính năng theo nhóm người dùng hoặc quyền truy cập; Ops Flag và Kill Switch giúp đội vận hành nhanh chóng vô hiệu hóa một chức năng khi phát hiện lỗi, suy giảm hiệu năng hoặc áp lực tải bất thường. Việc phân loại này cho thấy Feature Flag không chỉ là một câu lệnh điều kiện trong mã nguồn mà còn là một công cụ hỗ trợ quản lý phát hành, thử nghiệm sản phẩm và vận hành hệ thống. [1]

Tuy nhiên, khi số lượng flag, môi trường và điều kiện đánh giá tăng lên, việc quản lý bằng biến môi trường hoặc tệp cấu hình riêng lẻ trở nên khó kiểm soát. Trạng thái flag có thể không nhất quán giữa các máy chủ; người quản trị khó xác định ai đã thay đổi cấu hình; các ứng dụng phải tự cài đặt logic đánh giá; và những flag đã hết mục đích có thể tiếp tục tồn tại, tạo ra technical debt. Nghiên cứu thực nghiệm về việc sử dụng Feature Toggle trong doanh nghiệp cũng ghi nhận vai trò của các công cụ quản lý chuyên dụng, giá trị mặc định, metadata và nhật ký thay đổi đối với quá trình vận hành flag an toàn.

Những hạn chế trên tạo ra nhu cầu xây dựng một nền tảng Feature Flag tập trung. Thay vì để từng ứng dụng tự lưu trữ cấu hình và tự quyết định cách đánh giá flag, nền tảng cung cấp một nguồn cấu hình thống nhất, giao diện quản trị, cơ chế đánh giá có quy tắc, API tích hợp

và lịch sử thay đổi. Mô hình này giúp quá trình phát hành trở nên có kiểm soát, có khả năng truy vết và dễ mở rộng hơn khi số lượng ứng dụng hoặc đội phát triển tăng lên.

Xuất phát từ bối cảnh đó, đề tài tập trung nghiên cứu và xây dựng một nền tảng quản lý Feature Flag dành cho ứng dụng web. Hệ thống bao gồm dashboard quản trị để tạo và cấu hình flag; rule engine để đánh giá theo người dùng, vai trò và tỷ lệ rollout; evaluation API để các ứng dụng xác định trạng thái tính năng; audit log để ghi nhận thay đổi; cùng demo app nhằm minh họa quá trình tích hợp và sử dụng Feature Flag trong thực tế.

Việc thực hiện đề tài không chỉ nhằm xây dựng một chức năng bật hoặc tắt tính năng. Mục tiêu quan trọng hơn là nghiên cứu cách thiết kế một hệ thống có thể đưa ra quyết định nhất quán, xác định rõ thứ tự ưu tiên giữa các quy tắc, xử lý an toàn khi thiếu dữ liệu hoặc xảy ra lỗi, đồng thời cân bằng giữa tính linh hoạt, hiệu năng, khả năng kiểm soát và mức độ phức tạp của kiến trúc. Thông qua quá trình phân tích, thiết kế, triển khai và kiểm thử, đề tài hướng đến việc làm rõ giá trị của Feature Flag trong vòng đời phát triển phần mềm hiện đại, đồng thời thể hiện khả năng giải quyết vấn đề, tư duy thiết kế và khả năng giải thích các quyết định công nghệ được sử dụng trong hệ thống.

Tóm tắt nội dung và đóng góp

Báo cáo trình bày quá trình nghiên cứu Feature Flag và xây dựng một hệ thống quản lý phát hành tính năng an toàn cho ứng dụng web. Trọng tâm của đề tài là tách quá trình *deploy* mã nguồn khỏi quyết định *release* tính năng, qua đó hỗ trợ phát hành từng phần, targeting theo người dùng, rollback nhanh và truy vết thay đổi cấu hình.

Nội dung báo cáo gồm bốn phần chính: giới thiệu bài toán và phạm vi triển khai; trình bày cơ sở lý thuyết, khảo sát giải pháp hiện có, yêu cầu, kiến trúc, dữ liệu, API và Rule Evaluation Engine; mô tả kết quả xây dựng, kiểm thử, CI và các kịch bản demo; cuối cùng là tổng kết, hạn chế và hướng phát triển.

Sản phẩm được triển khai dưới dạng npm workspace gồm Backend API, Admin Dashboard, Demo Application, JavaScript SDK và PostgreSQL. Backend sử dụng NestJS, Prisma và PostgreSQL để quản lý project, feature flag, rule, flag group, audit log, history, statistics và Evaluation API. Dashboard hỗ trợ cấu hình và quan sát hệ thống; Demo Application dùng SDK để gọi Evaluation API, thay đổi giao diện theo kết quả flag và bổ sung demo backend API để chứng minh tính năng cũng bị chặn ở phía server khi flag tắt.

Rule Evaluation Engine được thiết kế với thứ tự ưu tiên rõ ràng: archived flag, disabled config, group kill switch, flag kill switch, global on, các rule targeting và default off. Hệ thống hỗ trợ user allowlist, role targeting, percentage rollout bằng stable hashing, reason code ổn định và mặc định fail-closed khi không thể đánh giá an toàn.

Các đóng góp chính của đề tài bao gồm:

- Hệ thống hóa kiến thức về Feature Flag, deploy/release separation, rollout strategy, kill switch, audit log, caching, consistency và bảo mật evaluation endpoint.
- Thiết kế kiến trúc tách rõ control plane và data plane, giúp phân biệt phần quản trị cấu

hình với luồng evaluation runtime.

- Xây dựng luồng end-to-end gồm Backend API, Admin Dashboard, Demo Application, JavaScript SDK, PostgreSQL migration/seed và Docker Compose.
- Bổ sung demo backend API cho Demo Application với các endpoint GET tương ứng từng feature flag, dùng JavaScript SDK ở phía server để fail-closed trước khi trả dữ liệu tính năng.
- Hiện thực Evaluation Engine xác định, fail-closed, hỗ trợ rule targeting, percentage rollout ổn định và group/flag kill switch.
- Triển khai audit log append-only với before/after snapshot, đồng thời dùng audit log làm nguồn cho configuration history.
- Bổ sung các phần mở rộng gồm evaluation snapshot cache, RBAC demo, statistics dashboard, JavaScript SDK fail-closed, GitHub Actions CI và bộ kiểm thử tự động.

Kết quả thực hiện cho thấy hệ thống đáp ứng mục tiêu mini project: có báo cáo nghiên cứu, backend API, dashboard, demo app, SDK, lưu trữ bền vững, validation, xử lý lỗi, audit log, seed data, kiểm thử và quy trình chạy thử. Dù chưa phải nền tảng thương mại hoàn chỉnh, sản phẩm đã minh họa được các nguyên lý quan trọng của Feature Flag và tạo nền tảng để mở rộng trong tương lai.

Mục lục

1 Giới thiệu **1**

1.1 Đặt vấn đề 1

1.2 Mục tiêu của báo cáo 2

1.3 Câu hỏi nghiên cứu 2

1.4 Phạm vi triển khai 3

2 Nội dung **4**

2.1 Tổng quan về Feature Flag 4

2.1.1 Khái niệm Feature Flag 4

2.1.2 Deploy và Release 5

2.1.3 Vai trò trong vòng đời phần mềm 5

2.1.4 Phân loại flag và chiến lược phát hành 6

2.1.5 Lợi ích và rủi ro 6

2.2 Khảo sát các nền tảng Feature Flag hiện có 7

2.2.1 Nền tảng LaunchDarkly 7

2.2.2 Nền tảng Unleash 7

2.2.3 So sánh và bài học rút ra 7

2.2.4 Định hướng của hệ thống đề xuất 8

2.3 Phân tích yêu cầu hệ thống 9

2.3.1 Yêu cầu chức năng 9

2.3.3 Yêu cầu phi chức năng 11

2.3.4 Đặc tả use case 13

2.4 Kiến trúc tổng thể của hệ thống 17

2.4.1 Các thành phần chính 18

2.4.2 Control plane và data plane . . . 19
2.4.3 Luồng quản trị Feature Flag . . . 20
2.4.4 Luồng đánh giá Feature Flag . . . 21
2.4.5 Luồng tích hợp ứng dụng khách . . . 22
2.4.6 Lựa chọn công nghệ . . . 22
2.5 Thiết kế dữ liệu . . . 22
2.5.1 Mô hình dữ liệu tổng thể . . . 23
2.5.2 Chuẩn hóa dữ liệu . . . 29
2.5.3 Ràng buộc toàn vẹn dữ liệu . . . 30
2.6 Thiết kế Rule Evaluation Engine . . . 32
2.6.1 Contract đầu vào và evaluation context . . . 32
2.6.2 Snapshot cấu hình và luồng thực thi data plane . . . 33
2.6.3 Thứ tự ưu tiên khi đánh giá . . . 34
2.6.4 Rule targeting và stable percentage rollout . . . 35
2.6.5 EvaluationResult, reason code và fallback . . . 36
2.6.6 Khả năng mở rộng và kiểm chứng . . . 38
2.7 Thiết kế API . . . 39
2.7.1 Control plane API . . . 39
2.7.2 Data plane API . . . 41
2.7.3 Monitoring API . . . 43
2.7.4 Validation và mô hình lỗi . . . 44
2.8 Hiệu năng, nhất quán và bảo mật . . . 45
2.8.1 Hiệu năng data plane: snapshot cache, TTL và thống kê . . . 45
2.8.2 Cache key, invalidation và tính nhất quán cấu hình . . . 46
2.8.3 Trust boundary của evaluation context và SDK . . . 47
2.8.4 Bảo vệ Management API bằng RBAC và validation . . . 49
2.8.5 Auditability và khả năng truy vết thay đổi . . . 49

3 Kết quả thực hiện và đánh giá 51

3.1 Môi trường và cấu trúc triển khai . . . 51
3.1.1 Cấu trúc mã nguồn . . . 51
3.1.2 Môi trường phát triển . . . 52

3.1.3 Database migration và seed data . . . 52
3.1.4 Triển khai bằng Docker Compose . . . 52
3.1.5 Quy trình khởi chạy hệ thống . . . 53
3.2 Kết quả xây dựng . . . 54
3.2.1 Backend API . . . 54
3.2.2 Frontend Dashboard . . . 55
3.2.3 Demo Application . . . 55
3.3 Kiểm thử và đánh giá . . . 56
3.3.1 Unit test . . . 56
3.3.2 Integration test và E2E test . . . 57
3.3.3 Luồng CI trong CI/CD . . . 57
3.4 Các kịch bản demo . . . 58
3.4.1 Quan sát project, flag và ứng dụng demo . . . 58
3.4.2 Bật và tắt tính năng toàn cục . . . 58
3.4.3 Targeting theo role . . . 58
3.4.4 Rollout theo phần trăm . . . 58
3.4.5 Kill switch cấp flag và group . . . 59
3.4.6 Backend API guard cho tính năng . . . 59
3.4.7 Audit log, history và statistics . . . 59
3.4.8 Fallback của JavaScript SDK . . . 59
3.4.9 Phân quyền Admin, Developer và Viewer . . . 59

4 Kết luận **60**

4.1 Tổng kết . . . 60
4.1.1 Kết quả và đóng góp . . . 60
4.1.2 Bài học kinh nghiệm . . . 61
4.2 Hạn chế và hướng phát triển . . . 61
4.2.1 Hạn chế . . . 61
4.2.2 Hướng phát triển . . . 62

Tài liệu tham khảo **64**

Danh sách hình vẽ

2.1 Use case model diagram của Feature Flag Platform 13
2.2 Kiến trúc tổng thể của hệ thống Feature Flag Platform 18
2.3 Phân tách control plane và data plane trong hệ thống 20
2.4 Luồng đánh giá Feature Flag từ ứng dụng khách đến Rule Evaluation Engine . . 21
2.5 Sơ đồ thực thể liên kết rút gọn của hệ thống Feature Flag Platform 23
2.6 Flowchart thứ tự ưu tiên của Rule Evaluation Engine 34
2.7 Luồng cache invalidation và bảo đảm tính nhất quán cấu hình 47

3.1 Các service chính trong quy trình triển khai bằng Docker Compose 53
3.2 Minh chứng kết quả xây dựng Backend API 54
3.3 Minh chứng kết quả xây dựng Frontend Dashboard 55
3.4 Minh chứng kết quả xây dựng Demo Application 56

Danh sách bảng

2.1 So sánh deploy và release 5
2.2 Đặc điểm vòng đời của một số loại Feature Flag 6
2.3 Tổng hợp so sánh LaunchDarkly và Unleash 8
2.4 Yêu cầu chức năng mức MVP 10
2.5 Yêu cầu chức năng mức khuyến khích 11
2.6 Đặc tả use case Quản lý project 14
2.7 Đặc tả use case Quản lý Feature Flag 14
2.8 Đặc tả use case Cấu hình rule rollout 15
2.9 Đặc tả use case Đánh giá Feature Flag 15
2.10 Đặc tả use case Xem audit log và lịch sử cấu hình 16
2.11 Đặc tả use case Quản lý quyền truy cập demo 16
2.12 Đặc tả use case Theo dõi thống kê evaluation 17
2.13 Đặc tả use case Kích hoạt group kill switch 17
2.14 Mô tả dữ liệu bảng `projects` 24
2.15 Mô tả dữ liệu bảng `environments` 24
2.16 Mô tả dữ liệu bảng `feature_flags` 25
2.17 Mô tả dữ liệu bảng `flag_groups` 25
2.18 Mô tả dữ liệu bảng `flag_group_configs` 26
2.19 Mô tả dữ liệu bảng `flag_environment_configs` 26
2.20 Mô tả dữ liệu bảng `flag_rules` 27
2.21 Mô tả dữ liệu bảng `flag_evaluation_metrics` 28
2.22 Mô tả dữ liệu bảng `sample_user_contexts` 28
2.23 Mô tả dữ liệu bảng `audit_log_entries` 29
2.24 Các ràng buộc duy nhất chính trong mô hình dữ liệu 30

2.25 Tương tác giữa trạng thái và serving mode 35
2.26 Reason code cho trạng thái hệ thống 37
2.27 Quan hệ giữa mã lý do và matched rule 37
2.28 Đặc tả Project API 39
2.29 Đặc tả Feature Flag API 40
2.30 Đặc tả Rule API 40
2.31 Đặc tả Flag Group API 41
2.32 Đặc tả Sample User API 41
2.33 Đặc tả Evaluation API 41
2.34 Đặc tả Audit Log và Flag History API 43
2.35 Đặc tả Evaluation Statistics API 44

3.1 Thống kê unit test đã triển khai 57

Danh sách đoạn mã

2.1 Điều khiển hành vi ứng dụng bằng Feature Flag 4
2.2 Cấu trúc evaluation request 32
2.3 Thứ tự ưu tiên của Rule Evaluation Engine 34
2.4 Tạo giá trị băm ổn định bằng SHA-256 36
2.5 Chuyển đổi SHA-256 digest thành bucket phần trăm 36
2.6 Cấu trúc EvaluationResult 36
2.7 Tập reason code của Evaluation API 37
2.8 Kết quả fallback mặc định 38
2.9 Xử lý lỗi nội bộ của Evaluation Engine 38
2.10 Ví dụ request Evaluation API 42
2.11 Ví dụ response Evaluation API 42
2.12 Định dạng key nghiệp vụ 44
2.13 Mẫu error response của control plane 44

Chương 1

Giới thiệu

1.1 Đặt vấn đề

Trong phát triển phần mềm hiện đại, triển khai mã nguồn và phát hành tính năng không nên luôn diễn ra đồng thời. Một tính năng có thể đã được deploy lên môi trường production nhưng chỉ nên mở cho một nhóm người dùng nhỏ, một vai trò cụ thể hoặc một tỷ lệ người dùng nhất định. Nếu phát hành trực tiếp cho toàn bộ người dùng, lỗi nghiệp vụ, lỗi hiệu năng hoặc hành vi ngoài dự kiến có thể gây ảnh hưởng diện rộng và buộc đội phát triển phải rollback hoặc deploy lại.

Feature Flag giải quyết bài toán này bằng cách đưa quyết định bật/tắt tính năng ra khỏi mã nguồn đã triển khai. Thay vì thay đổi ứng dụng mỗi khi cần release hoặc rollback, đội vận hành có thể cập nhật cấu hình flag, rule và kill switch. Tuy nhiên, nếu các flag chỉ được quản lý rời rạc trong biến môi trường hoặc logic riêng của từng ứng dụng, hệ thống sẽ khó kiểm soát lịch sử thay đổi, khó bảo đảm kết quả nhất quán và khó mở rộng khi số lượng flag tăng.

Vì vậy, đề tài lựa chọn xây dựng một nền tảng Feature Flag thu nhỏ cho ứng dụng web. Hệ thống tập trung vào các năng lực cốt lõi: quản lý cấu hình tập trung, đánh giá flag theo context người dùng, rollout xác định, rollback nhanh, audit log và minh họa trực tiếp qua một demo application.

1.2 Mục tiêu của báo cáo

Báo cáo hướng đến hai mục tiêu chính: nghiên cứu nguyên lý Feature Flag và trình bày kết quả xây dựng một hệ thống minh họa có thể chạy được.

Về nghiên cứu, báo cáo làm rõ Feature Flag là gì, vì sao kỹ thuật này giúp tách deploy khỏi release, các loại flag phổ biến, các chiến lược rollout như global on/off, user allowlist, role targeting và percentage rollout, cũng như các yêu cầu vận hành như kill switch, audit log, caching, consistency, safe defaults và bảo mật endpoint evaluation.

Về triển khai, đề tài xây dựng một platform gồm Backend API NestJS, cơ sở dữ liệu PostgreSQL qua Prisma, Admin Dashboard React, Demo Application React và JavaScript SDK. Hệ thống cho phép tạo project, quản lý feature flag, cấu hình rule, gọi Evaluation API, ghi audit log, xem history/statistics và trình diễn các kịch bản release an toàn. Các quyết định kỹ thuật được giải thích theo hướng bảo đảm deterministic evaluation, fail-closed behavior, control-plane/data-plane separation và khả năng kiểm thử.

1.3 Câu hỏi nghiên cứu

Báo cáo tập trung trả lời các câu hỏi sau:

1. Feature Flag giúp giảm rủi ro trong quy trình phát hành phần mềm như thế nào?
2. Làm thế nào để tách việc deploy mã nguồn khỏi quyết định release tính năng cho người dùng?
3. Một nền tảng Feature Flag tối thiểu cần những thành phần nào ở backend, dashboard, SDK, demo app và cơ sở dữ liệu?
4. Evaluation Engine cần thứ tự ưu tiên và reason code như thế nào để kết quả bật/tắt rõ ràng, nhất quán và dễ kiểm thử?
5. Làm thế nào để percentage rollout ổn định cho cùng một người dùng mà không lưu dữ liệu cá nhân nhạy cảm?
6. Hệ thống cần xử lý ra sao khi project/flag không tồn tại, context không hợp lệ hoặc Evaluation API/SDK gặp lỗi?
7. Kiến trúc và công nghệ nào phù hợp cho một mini project nhưng vẫn thể hiện được tư

duy thiết kế hệ thống và khả năng mở rộng?

1.4 Phạm vi triển khai

Phạm vi triển khai của đề tài là một Feature Flag Platform phục vụ demo và nghiên cứu, không phải một sản phẩm thương mại đầy đủ. Hệ thống được tổ chức dưới dạng npm workspace với các thành phần chính sau:

- **Backend API**: cung cấp REST API cho project, feature flag, rule, flag group, evaluation, audit log, history, statistics, sample user và RBAC demo.
- **Rule Evaluation Engine**: đánh giá flag theo lifecycle, trạng thái cấu hình, group/flag kill switch, global on, ordered rules và default off.
- **Admin Dashboard**: hỗ trợ quản trị project, flag, rule, group kill switch, audit log, history, statistics và hiển thị trạng thái theo quyền người dùng.
- **Demo Application**: mô phỏng ứng dụng web tích hợp Feature Flag, gồm storefront React và một demo backend nhỏ. Frontend gọi SDK để hiển thị hoặc ẩn tính năng theo kết quả evaluation; demo backend cung cấp các endpoint được guard bởi cùng quyết định evaluation để chứng minh API phía server cũng bị chặn khi flag tắt.
- **JavaScript SDK**: cung cấp hàm `evaluate`, `isEnabled`, `getVariant`, validate response, xử lý timeout và fallback fail-closed.
- **Persistence và delivery**: dùng PostgreSQL, Prisma migration/seed, Jest/Supertest, GitHub Actions CI và Docker Compose cho môi trường chạy thử.

Các chức năng được hỗ trợ gồm global enable/disable, user allowlist, role targeting, percentage rollout, audit log append-only, audit-backed history, evaluation statistics, group kill switch, RBAC demo và cache snapshot evaluation. Những vấn đề production-scale như multi-region deployment, streaming SDK, approval workflow hoàn chỉnh, A/B testing nâng cao, monitoring hạ tầng chuyên sâu, backup/recovery và xử lý lưu lượng rất lớn được xem là hướng phát triển sau phạm vi mini project.

Chương 2

Nội dung

2.1 Tổng quan về Feature Flag

2.1.1 Khái niệm Feature Flag

Feature Flag, còn gọi là Feature Toggle hoặc Feature Switch, là kỹ thuật cho phép phần mềm quyết định tại thời điểm chạy liệu một tính năng hoặc một nhánh xử lý có được kích hoạt hay không. Quyết định này dựa trên cấu hình bên ngoài mã nguồn và evaluation context của đối tượng đang sử dụng hệ thống [1, 2].

Một flag thường có các thành phần cốt lõi: khóa định danh, trạng thái hoặc cấu hình mặc định, rule targeting, chiến lược rollout, evaluation context và giá trị trả về. Trong một nền tảng quản lý tập trung, các flag còn được đặt trong phạm vi project và environment để cùng một tính năng có thể có cấu hình khác nhau giữa development, staging và production [3].

Ở runtime, ứng dụng gọi SDK hoặc Evaluation API bằng flag key và context. Kết quả trả về được dùng để chọn nhánh xử lý trong mã nguồn đã deploy:

Đoạn mã 2.1: Điều khiển hành vi ứng dụng bằng Feature Flag

```
const result = await featureFlagClient.evaluate(
    "sample-feature",
    userContext
);

if (result.enabled) {
```

```
    return showFlaggedFeature();
}

return showDefaultExperience();
```

Điểm quan trọng là Feature Flag không tự thay đổi mã nguồn; nó chỉ thay đổi cấu hình quyết định nhánh xử lý nào được sử dụng.

2.1.2 Deploy và Release

Deploy là quá trình đưa phiên bản phần mềm lên môi trường chạy. Release là quyết định cho phép người dùng thực sự sử dụng một tính năng. Feature Flag có giá trị vì nó tách hai hoạt động này: mã có thể đã được deploy nhưng tính năng vẫn chưa được release rộng rãi [4].

Sự khác biệt giữa deploy và release được thể hiện trong Bảng 2.1.

Bảng 2.1: So sánh deploy và release

Tiêu chí	**Deploy**	**Release**
Mục tiêu	Đưa mã nguồn lên môi trường chạy	Cung cấp tính năng cho người dùng
Phạm vi tác động	Hạ tầng, dịch vụ, phiên bản ứng dụng	Người dùng hoặc nhóm người dùng
Cách thực hiện	CI/CD, script triển khai, hạ tầng	Cấu hình flag hoặc rule targeting
Đảo ngược	Có thể cần rollback phiên bản	Có thể tắt hoặc thu hẹp flag

Khi deploy và release bị gắn chặt, mỗi lần đưa mã lên production đều có thể làm thay đổi hành vi của toàn bộ hệ thống. Khi tách chúng bằng Feature Flag, đội phát triển có thể kiểm thử có kiểm soát, mở rộng dần phạm vi phát hành và tắt nhanh tính năng nếu phát sinh sự cố [1].

2.1.3 Vai trò trong vòng đời phần mềm

Trong vòng đời phần mềm, Feature Flag được dùng như một cơ chế điều phối release chứ không chỉ là câu lệnh điều kiện. Các vai trò chính gồm:

- tích hợp sớm mã chưa hoàn thiện vào nhánh chính mà chưa hiển thị cho người dùng;
- kiểm thử tính năng trên môi trường thật với nhóm nội bộ hoặc khách hàng beta;
- mở rộng phát hành theo từng giai đoạn thay vì bật cho toàn bộ người dùng ngay lập tức;

- rollback runtime bằng cách tắt flag khi tính năng gây lỗi hoặc ảnh hưởng hiệu năng;
- ghi nhận lịch sử cấu hình để truy vết nguyên nhân khi hành vi ứng dụng thay đổi.

Các vai trò trên là cơ sở để mini project tập trung vào quản lý cấu hình, evaluation có precedence rõ ràng, audit log và demo runtime thay vì chỉ xây dựng chức năng bật/tắt đơn giản.

2.1.4 Phân loại flag và chiến lược phát hành

Các loại flag phổ biến khác nhau ở mục đích và thời gian tồn tại. Release flag thường ngắn hạn và phục vụ phát hành tính năng mới. Experiment flag dùng cho thử nghiệm sản phẩm. Ops hoặc kill-switch flag phục vụ vận hành và xử lý sự cố. Permission flag giới hạn tính năng theo nhóm người dùng hoặc gói dịch vụ [1, 5].

Bảng 2.2: Đặc điểm vòng đời của một số loại Feature Flag

Loại flag	**Mục đích chính**	**Thời gian tồn tại**	**Rủi ro chính**
Release flag	Phát hành tính năng mới	Ngắn hạn	Quên dọn dẹp
Experiment flag	A/B testing hoặc đo lường	Theo thời gian thử nghiệm	Sai lệch dữ liệu
Ops/Kill switch	Tắt nhanh khi có sự cố	Dài hơn, tùy vận hành	Bị lạm dụng thay cấu hình
Permission flag	Giới hạn theo nhóm người dùng	Theo chính sách sản phẩm	Nhầm với authorization

Về chiến lược release, hệ thống mini project tập trung vào bốn trường hợp đủ để minh họa giá trị cốt lõi: bật/tắt toàn cục, user allowlist, role targeting và percentage rollout. Percentage rollout cần stable hashing để cùng một targeting key nhận kết quả ổn định khi cấu hình không đổi [6, 7].

2.1.5 Lợi ích và rủi ro

Lợi ích chính của Feature Flag là giảm rủi ro phát hành: đội phát triển có thể deploy thường xuyên, release có kiểm soát, rollback nhanh và quan sát tác động qua audit/statistics. Tuy nhiên, cùng một cơ chế cũng tạo thêm trách nhiệm vận hành.

Các rủi ro cần quản lý gồm feature flag debt, cấu hình sai, rule chồng chéo khó giải thích, context chứa dữ liệu nhạy cảm và việc dùng flag thay cho cơ chế authorization thật. Vì vậy, nền tảng Feature Flag cần validation, precedence rõ ràng, safe default, audit log, phân quyền quản trị và quy trình dọn dẹp flag sau khi hết mục đích [8, 9, 10].

2.2 Khảo sát các nền tảng Feature Flag hiện có

Phần này khảo sát hai nền tảng tiêu biểu là LaunchDarkly và Unleash nhằm rút ra các nguyên tắc thiết kế cần kế thừa cho mini project. Nội dung chỉ tập trung vào điểm khác biệt và bài học thiết kế, tránh lặp lại các khái niệm Feature Flag đã trình bày ở phần tổng quan.

2.2.1 Nền tảng LaunchDarkly

LaunchDarkly là nền tảng Feature Flag thương mại dạng dịch vụ được quản lý. Nền tảng tổ chức tài nguyên theo project, environment và flag; cung cấp SDK đa nền tảng; hỗ trợ targeting rule có thứ tự, percentage rollout, nhiều variation, audit log và quản trị quyền truy cập [3, 6].

Điểm đáng chú ý đối với mini project là mô hình ordered rule. Người quản trị có thể đặt rule cụ thể trước rule tổng quát, ví dụ user allowlist trước percentage rollout. Cách biểu diễn này giúp kết quả evaluation dễ giải thích vì engine dừng tại rule đầu tiên khớp.

Hạn chế chính khi tham chiếu LaunchDarkly cho mini project là phạm vi tính năng rất rộng. Các khả năng như segment phức tạp, nhiều variation, workflow phê duyệt và phân tích thử nghiệm chuyên sâu được xem là hướng mở rộng, không phải yêu cầu triển khai ban đầu.

2.2.2 Nền tảng Unleash

Unleash là nền tảng Feature Flag mã nguồn mở, có thể self-hosted hoặc dùng dịch vụ được quản lý. Nền tảng sử dụng activation strategy để quyết định flag có được bật hay không trong từng environment; SDK thường tải cấu hình về và đánh giá tại ứng dụng [2, 5].

Điểm đáng chú ý đối với mini project là khả năng tự triển khai và tư duy tách control plane khỏi runtime evaluation. Unleash cũng nhấn mạnh lifecycle của flag, metric evaluation và việc tránh để hệ thống Feature Flag trở thành phụ thuộc đồng bộ bắt buộc của mọi request nghiệp vụ.

Hạn chế của mô hình self-hosted là tổ chức phải tự chịu trách nhiệm về database, backup, monitoring, nâng cấp và mở rộng. Vì mini project cần dễ chạy và dễ trình diễn, hệ thống chỉ mô phỏng phần lõi bằng một backend duy nhất, PostgreSQL và cache đơn giản.

2.2.3 So sánh và bài học rút ra

Bảng 2.3 tóm tắt các điểm so sánh có ảnh hưởng trực tiếp đến thiết kế mini project.

Bảng 2.3: Tổng hợp so sánh LaunchDarkly và Unleash

Tiêu chí	**LaunchDarkly**	**Unleash**
Mô hình triển khai	Dịch vụ được quản lý	Managed hoặc self-hosted
Đơn vị logic	Ordered targeting rule	Activation strategy và constraint
Evaluation	Chủ yếu qua SDK	Chủ yếu qua SDK hoặc proxy/edge
Rollout	Percentage rollout ổn định	Gradual rollout ổn định
Audit/RBAC	Tích hợp phong phú	Phụ thuộc phiên bản/mô hình triển khai
Bài học cho mini project	Precedence rõ, reason dễ giải thích	Tách plane, self-hosted workflow, lifecycle

Từ khảo sát, mini project kế thừa các nguyên tắc sau:

- tổ chức tài nguyên theo project, environment và flag;
- tách control plane quản trị khỏi data plane evaluation;
- dùng evaluation context thay vì hard-code điều kiện trong ứng dụng;
- áp dụng stable rollout bằng khóa không chứa PII;
- trả reason code để giải thích kết quả evaluation;
- ghi audit log cho mọi mutation quan trọng;
- cung cấp SDK và fallback an toàn cho ứng dụng tích hợp.

2.2.4 Định hướng của hệ thống đề xuất

Hệ thống đề xuất không cố gắng sao chép đầy đủ LaunchDarkly hoặc Unleash. Phạm vi được chọn là một lát cắt đủ nhỏ để triển khai trong mini project nhưng vẫn thể hiện được bản chất của nền tảng Feature Flag: quản lý cấu hình, đánh giá runtime, thay đổi hành vi ứng dụng, auditability và trình diễn release an toàn.

Các chức năng được kế thừa trực tiếp gồm project/environment, flag config theo environment, ordered rule, user/role targeting, percentage rollout, kill switch, audit log, statistics, SDK và Docker Compose. Các chức năng được đơn giản hóa gồm multi-variant testing, segment phức tạp, approval workflow, realtime streaming, multi-region deployment và phân quyền enterprise.

Điểm khác biệt của MiniProject là ưu tiên tính minh bạch và khả năng giải thích hơn độ rộng

tính năng. Vì vậy, Rule Evaluation Engine dùng precedence cố định, response có reason code rõ ràng, audit log append-only và Dashboard tập trung vào các kịch bản demo quan trọng nhất.

2.3 Phân tích yêu cầu hệ thống

Mục này chuyển các yêu cầu của đề tài thành phạm vi hệ thống cần xây dựng. Hệ thống được xem như một nền tảng Feature Flag thu nhỏ cho ứng dụng web, gồm phần quản trị cấu hình, phần đánh giá flag ở thời điểm chạy và ứng dụng demo chứng minh giá trị của việc tách *deploy* khỏi *release*.

Các stakeholder và actor chính của hệ thống gồm:

- **Quản trị viên**: chịu trách nhiệm tạo project, quản lý Feature Flag, cấu hình rule, kích hoạt kill switch, phân quyền demo và theo dõi toàn bộ lịch sử thay đổi.
- **Nhà phát triển**: chuẩn bị cấu hình rollout, kiểm thử rule, tích hợp Evaluation API hoặc JavaScript SDK vào ứng dụng khách và sử dụng audit/history để đối chiếu khi có sự cố.
- **Người xem**: quan sát project, flag, audit log, lịch sử cấu hình và thống kê evaluation nhưng không được thay đổi cấu hình.
- **Ứng dụng tích hợp Feature Flag**: gửi evaluation request kèm context người dùng không nhạy cảm và dùng kết quả trả về để bật hoặc ẩn tính năng tại runtime.
- **Người dùng cuối của ứng dụng demo**: không trực tiếp quản trị nền tảng nhưng là đối tượng nhìn thấy tác động của các quyết định bật/tắt, targeting hoặc percentage rollout.

Từ các actor trên, các trường hợp sử dụng chính được gom thành bốn nhóm. Nhóm quản trị cấu hình bao gồm quản lý project, Feature Flag, rule rollout và kill switch. Nhóm runtime evaluation bao gồm việc ứng dụng khách gọi Evaluation API hoặc SDK để nhận quyết định bật/tắt có reason code. Nhóm quan sát và truy vết bao gồm audit log, lịch sử cấu hình và thống kê số lượt evaluation. Nhóm trình diễn bao gồm các kịch bản global enable/disable, targeting theo role hoặc user, percentage rollout và rollback nhanh mà không cần triển khai lại ứng dụng.

2.3.1 Yêu cầu chức năng

Bảng 2.4 tóm tắt các yêu cầu chức năng ở mức MVP. Đây là phạm vi bắt buộc để hệ thống có một luồng end-to-end hoàn chỉnh từ quản trị flag đến ứng dụng demo.

Bảng 2.4: Yêu cầu chức năng mức MVP

Mã	Yêu cầu	Actor chính	Kết quả mong đợi
FR-MVP-01	Quản lý project dùng làm phạm vi chứa Feature Flag và dữ liệu cấu hình liên quan.	Quản trị viên, nhà phát triển	Có thể tạo, xem và chọn project để quản trị flag.
FR-MVP-02	Quản lý vòng đời Feature Flag gồm tạo, xem danh sách, cập nhật thông tin và thể hiện trạng thái rõ ràng.	Quản trị viên, nhà phát triển	Dashboard hiển thị được flag và trạng thái quản trị của flag.
FR-MVP-03	Cấu hình rule cho flag, bao gồm bật/tắt toàn cục, allowlist theo user ID, targeting theo role và rollout theo phần trăm người dùng.	Quản trị viên, nhà phát triển	Rule được lưu bền vững và được engine sử dụng khi evaluation.
FR-MVP-04	Cung cấp Evaluation API để ứng dụng khách gửi project key, flag key và user context.	Ứng dụng tích hợp Feature Flag	API trả về quyết định bật/tắt cho flag tương ứng.
FR-MVP-05	Trả response evaluation có cấu trúc nhất quán, tối thiểu gồm `enabled`, `reason`, `projectKey` và `flagKey`.	Ứng dụng tích hợp Feature Flag	Ứng dụng khách có thể giải thích lý do bật/tắt và xử lý fallback.
FR-MVP-06	Ghi audit log cho các thay đổi cấu hình flag và rule.	Quản trị viên, nhà phát triển, người xem	Có thể truy vết ai thay đổi gì, khi nào và giá trị trước/sau.
FR-MVP-07	Cung cấp Dashboard với các màn hình project list, flag list, tạo/sửa flag, rule configuration và audit log.	Quản trị viên, nhà phát triển, người xem	Người dùng thao tác được luồng quản trị chính qua giao diện web.
FR-MVP-08	Cung cấp Demo Application gọi Evaluation API và hiển thị hoặc ẩn tính năng dựa trên kết quả flag.	Người dùng cuối, ứng dụng tích hợp Feature Flag	Demo chứng minh thay đổi cấu hình không cần deploy lại ứng dụng.
FR-MVP-09	Hỗ trợ tối thiểu hai kịch bản demo: global enable/disable và targeting theo role hoặc percentage rollout.	Quản trị viên, người dùng cuối	Người chấm có thể quan sát trực tiếp giá trị của Feature Flag.
FR-MVP-10	Lưu trữ project, flag, rule, sample user context và audit log trong database; có validation, xử lý lỗi cơ bản, seed data và hướng dẫn chạy.	Toàn hệ thống	Hệ thống có thể khởi chạy lặp lại và phục vụ kiểm thử/chấm điểm.

Sau khi MVP ổn định, hệ thống bổ sung các yêu cầu khuyến khích trong Bảng 2.5. Các yêu cầu này làm sản phẩm gần hơn với một nền tảng Feature Flag thực tế nhưng không được làm suy giảm baseline MVP.

Bảng 2.5: Yêu cầu chức năng mức khuyến khích

Mã	Yêu cầu	Actor chính	Giá trị bổ sung
FR-REC-01	Cache evaluation snapshot bằng in-memory cache hoặc Redis để giảm truy vấn lặp trong Evaluation API.	Ứng dụng tích hợp Feature Flag	Tăng hiệu năng runtime nhưng không thay đổi tính đúng đắn.
FR-REC-02	Cung cấp JavaScript SDK để ứng dụng web gọi Evaluation API qua giao diện ổn định, có timeout và fallback an toàn.	Nhà phát triển, ứng dụng tích hợp Feature Flag	Giảm mã tích hợp lặp lại ở phía client.
FR-REC-03	Bổ sung unit test cho Rule Evaluation Engine, đặc biệt là precedence, stable rollout, kill switch và trường hợp thiếu dữ liệu.	Nhà phát triển	Tăng độ tin cậy và khả năng bảo trì.
FR-REC-04	Cung cấp lịch sử cấu hình hoặc rule versioning dựa trên audit log.	Quản trị viên, nhà phát triển, người xem	Hỗ trợ truy vết và giải thích thay đổi theo thời gian.
FR-REC-05	Áp dụng RBAC demo với ba vai trò admin, developer và viewer cho các thao tác quản trị.	Quản trị viên, nhà phát triển, người xem	Thể hiện ranh giới quyền và giảm rủi ro thao tác nhầm.
FR-REC-06	Cung cấp statistics dashboard cho số lượt evaluation theo flag, kết quả và thời gian.	Quản trị viên, nhà phát triển, người xem	Hỗ trợ quan sát tác động của flag trong demo.
FR-REC-07	Hỗ trợ group kill switch để vô hiệu hóa nhanh một nhóm flag liên quan.	Quản trị viên, nhà phát triển	Cho phép rollback nhanh ở cấp nhóm tính năng.
FR-REC-08	Cung cấp Docker Compose để khởi chạy database, backend, dashboard, demo và các tác vụ migration/seed.	Nhà phát triển, người chấm	Đơn giản hóa setup và tăng khả năng tái lập khi trình diễn.

2.3.3 Yêu cầu phi chức năng

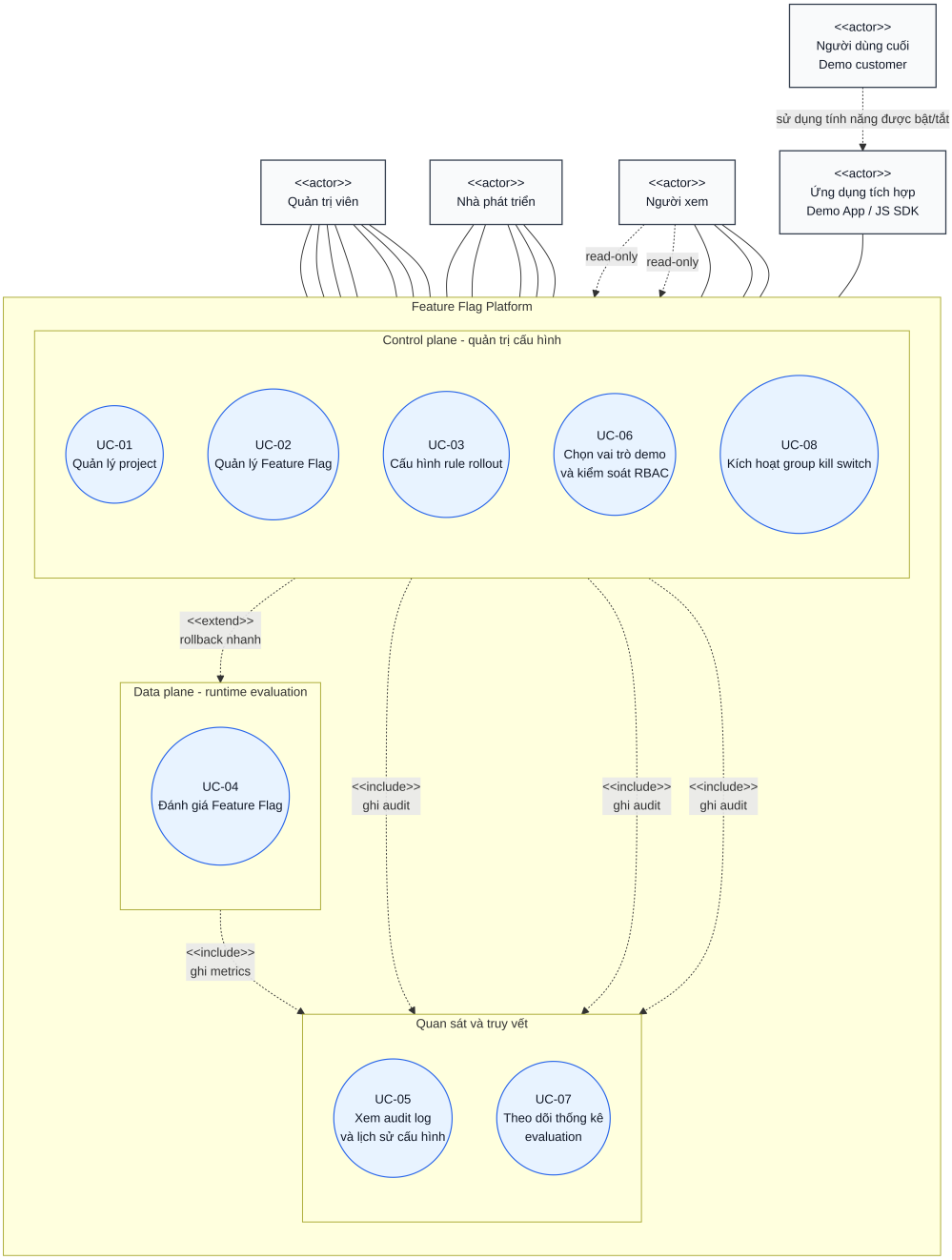
Các yêu cầu phi chức năng áp dụng cho cả MVP và các phần khuyến khích, nhằm đảm bảo hệ thống không chỉ có đủ chức năng mà còn có hành vi an toàn, dễ giải thích và phù hợp để trình diễn.

- **Tính đúng đắn và xác định**: cùng một cấu hình và cùng một evaluation context phải luôn cho cùng kết quả. Percentage rollout phải dùng stable hashing trên khóa ổn định, không chứa PII.
- **Mặc định an toàn**: khi project hoặc flag không tồn tại, hoặc khi evaluation không thể hoàn tất đáng tin cậy, hệ thống phải trả `enabled=false`; trường hợp không tìm thấy phải có reason `NOT_FOUND`.

- **Thứ tự ưu tiên rõ ràng**: engine phải tuân thủ precedence thống nhất: flag archived, cấu hình disabled, group kill switch, flag kill switch, global on, các rule targeting có thứ tự và cuối cùng là default off.
- **Tách control plane và data plane**: Dashboard và Management API quản lý cấu hình; Evaluation API và Rule Evaluation Engine phục vụ runtime decision. Ứng dụng khách không được phụ thuộc vào dữ liệu quản trị nội bộ.
- **Truy vết và nhất quán dữ liệu**: mutation quan trọng phải ghi audit log append-only với actor, timestamp, target và snapshot trước/sau trong cùng giao dịch lưu dữ liệu.
- **Hiệu năng evaluation**: Evaluation API cần độ trễ thấp; cache snapshot chỉ là tầng tăng tốc và có thể bị xóa mà không làm mất nguồn sự thật trong database.
- **Bảo mật và quyền truy cập**: Management API cần kiểm tra quyền theo RBAC demo; evaluation context do client gửi chỉ dùng để targeting, không được xem là bằng chứng phân quyền thật.
- **Không xem frontend hiding là lớp bảo vệ**: việc ẩn nút hoặc ẩn trang ở frontend chỉ cải thiện trải nghiệm người dùng; API phục vụ tính năng vẫn cần kiểm tra Feature Flag ở backend trước khi trả dữ liệu hoặc thực thi nghiệp vụ.
- **Bảo vệ dữ liệu cá nhân**: hệ thống ưu tiên stable targeting key và dữ liệu mẫu không nhạy cảm; rollout không phụ thuộc vào thông tin định danh trực tiếp của người dùng thật.
- **Khả dụng cho demo**: seed data, README và Docker Compose phải giúp người chấm khởi chạy lại hệ thống trong môi trường mới với ít bước thủ công.
- **Khả năng bảo trì và mở rộng**: logic evaluation phải tách khỏi controller, có test tự động và có thể mở rộng thêm rule, SDK hoặc backend triển khai khác trong tương lai.
- **Tính dễ dùng của giao diện**: Dashboard phải phân biệt trạng thái quản trị của flag như Enabled, Disabled, Archived với trạng thái runtime On/Off; các thao tác bị chặn bởi quyền phải được hiển thị rõ ràng.

2.3.4 Đặc tả use case

Hình 2.1 trình bày use case model diagram của hệ thống. Sơ đồ thể hiện quan hệ giữa các actor quản trị, ứng dụng tích hợp Feature Flag và các use case chính được đặc tả bên dưới.



Hình 2.1: Use case model diagram của Feature Flag Platform

Use case: Quản lý project

Bảng 2.6: Đặc tả use case Quản lý project

Thuộc tính	**Mô tả**
Use case ID	UC-01.
Actor chính	Quản trị viên, nhà phát triển.
Mục tiêu	Tạo và chọn project làm phạm vi quản lý Feature Flag.
Tiền điều kiện	Người dùng có quyền truy cập Dashboard.
Luồng chính	Người dùng mở danh sách project, tạo hoặc chọn project, hệ thống lưu dữ liệu và hiển thị project đang được quản trị.
Ngoại lệ	Dữ liệu không hợp lệ hoặc project key trùng thì hệ thống từ chối và trả thông báo lỗi.
Hậu điều kiện	Project tồn tại trong database và có thể được dùng cho flag, rule và evaluation.

Use case: Quản lý Feature Flag

Bảng 2.7: Đặc tả use case Quản lý Feature Flag

Thuộc tính	**Mô tả**
Use case ID	UC-02.
Actor chính	Quản trị viên, nhà phát triển.
Mục tiêu	Tạo, cập nhật và quan sát trạng thái Feature Flag trong một project.
Tiền điều kiện	Project đã tồn tại; người dùng có quyền mutation phù hợp.
Luồng chính	Người dùng nhập key, tên, mô tả và trạng thái; Backend kiểm tra dữ liệu, lưu flag và ghi audit log.
Ngoại lệ	Flag key trùng, dữ liệu thiếu hoặc người dùng không đủ quyền thì thao tác bị từ chối.
Hậu điều kiện	Feature Flag được lưu bền vững, hiển thị trên Dashboard và sẵn sàng cấu hình rule.

Use case: Cấu hình rule rollout

Bảng 2.8: Đặc tả use case Cấu hình rule rollout

Thuộc tính	**Mô tả**
Use case ID	UC-03.
Actor chính	Quản trị viên, nhà phát triển.
Mục tiêu	Xác định cách flag được bật theo global mode, user allowlist, role targeting hoặc percentage rollout.
Tiền điều kiện	Feature Flag đã tồn tại và chưa bị archived.
Luồng chính	Người dùng cấu hình serving mode và danh sách rule có thứ tự; hệ thống validate tham số, lưu rule, invalidation snapshot liên quan và ghi audit log.
Ngoại lệ	Rule không hợp lệ, phần trăm rollout ngoài khoảng cho phép hoặc thiếu quyền thì hệ thống trả lỗi.
Hậu điều kiện	Evaluation Engine sử dụng rule mới trong các request tiếp theo.

Use case: Đánh giá Feature Flag

Bảng 2.9: Đặc tả use case Đánh giá Feature Flag

Thuộc tính	**Mô tả**
Use case ID	UC-04.
Actor chính	Ứng dụng tích hợp Feature Flag.
Mục tiêu	Nhận quyết định bật/tắt flag cho một context tại runtime.
Tiền điều kiện	Ứng dụng biết project key, flag key và endpoint evaluation; context dùng stable key không nhạy cảm.
Luồng chính	Ứng dụng gọi Evaluation API hoặc SDK; engine tải snapshot, áp dụng precedence và trả `enabled`, `reason`, `projectKey`, `flagKey`.
Ngoại lệ	Project/flag không tồn tại hoặc lỗi evaluation thì hệ thống trả kết quả fail-closed phù hợp.
Hậu điều kiện	Ứng dụng sử dụng kết quả để bật hoặc ẩn tính năng; cấu hình hệ thống không bị thay đổi bởi request evaluation.

Use case: Xem audit log và lịch sử cấu hình

Bảng 2.10: Đặc tả use case Xem audit log và lịch sử cấu hình

Thuộc tính	Mô tả
Use case ID	UC-05.
Actor chính	Quản trị viên, nhà phát triển, người xem.
Mục tiêu	Truy vết các thay đổi cấu hình và giải thích vì sao trạng thái flag thay đổi theo thời gian.
Tiền điều kiện	Đã có mutation cấu hình hoặc seed data tạo audit entry.
Luồng chính	Người dùng mở màn hình audit/history, lọc theo project hoặc target; hệ thống hiển thị actor, action, timestamp và snapshot trước/sau.
Ngoại lệ	Nếu không có dữ liệu phù hợp, hệ thống hiển thị trạng thái rỗng thay vì lỗi.
Hậu điều kiện	Người dùng có bằng chứng truy vết để phục vụ debug, báo cáo và trình diễn.

Use case: Quản lý quyền truy cập demo

Bảng 2.11: Đặc tả use case Quản lý quyền truy cập demo

Thuộc tính	Mô tả
Use case ID	UC-06.
Actor chính	Quản trị viên, nhà phát triển, người xem.
Mục tiêu	Thể hiện sự khác biệt quyền giữa admin, developer và viewer trong Dashboard.
Tiền điều kiện	Hệ thống có danh sách identity demo hoặc bearer identity được backend phân giải.
Luồng chính	Người dùng chọn vai trò demo; Backend áp dụng permission matrix cho từng endpoint; Dashboard vô hiệu hóa hoặc ẩn thao tác không được phép.
Ngoại lệ	Nếu role không có quyền mutation, hệ thống trả lỗi authorization và không ghi thay đổi cấu hình.
Hậu điều kiện	Audit log ghi actor đáng tin cậy từ backend; người xem chỉ quan sát, không thể thay đổi dữ liệu.

Use case: Theo dõi thống kê evaluation

Bảng 2.12: Đặc tả use case Theo dõi thống kê evaluation

Thuộc tính	Mô tả
Use case ID	UC-07.
Actor chính	Quản trị viên, nhà phát triển, người xem.
Mục tiêu	Quan sát số lượt evaluation theo flag, kết quả và thời gian để thấy tác động runtime của cấu hình.
Tiền điều kiện	Demo Application hoặc client đã phát sinh evaluation request.
Luồng chính	Người dùng mở statistics dashboard; hệ thống đọc số liệu aggregate và hiển thị tổng lượt, lượt bật/tắt và phân bố theo reason hoặc khoảng thời gian.
Ngoại lệ	Khi chưa có số liệu, Dashboard hiển thị hướng dẫn chạy demo để tạo dữ liệu.
Hậu điều kiện	Người dùng có thêm bằng chứng trực quan cho quá trình rollout và rollback.

Use case: Kích hoạt group kill switch

Bảng 2.13: Đặc tả use case Kích hoạt group kill switch

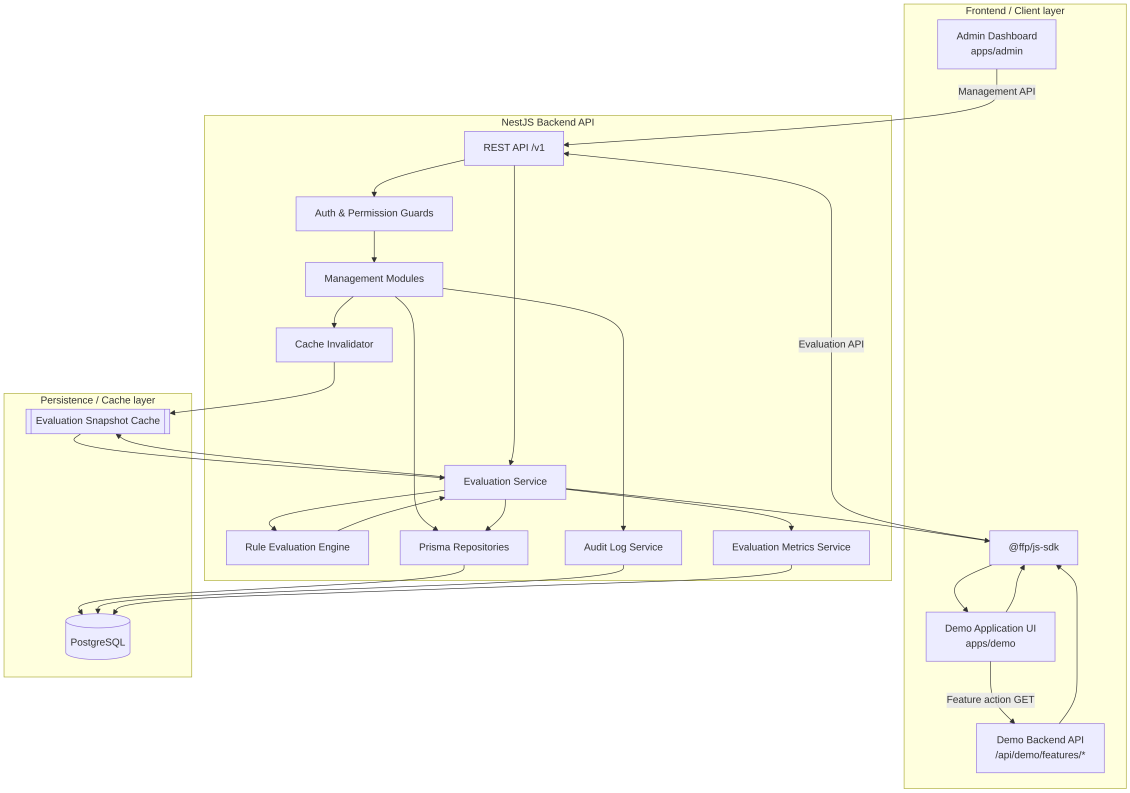
Thuộc tính	Mô tả
Use case ID	UC-08.
Actor chính	Quản trị viên, nhà phát triển.
Mục tiêu	Tắt nhanh một nhóm flag liên quan khi có sự cố hoặc cần rollback khẩn cấp.
Tiền điều kiện	Các flag đã được gán vào group; người dùng có quyền thay đổi kill switch.
Luồng chính	Người dùng bật kill switch cho group; hệ thống lưu trạng thái, ghi audit log và làm mới snapshot để evaluation sau đó trả về trạng thái tắt.
Ngoại lệ	Nếu group không tồn tại hoặc người dùng thiếu quyền, thao tác bị từ chối và không ảnh hưởng tới các flag.
Hậu điều kiện	Các flag thuộc group bị vô hiệu hóa ở runtime cho đến khi kill switch được gỡ.

2.4 Kiến trúc tổng thể của hệ thống

Phần này trình bày kiến trúc ở mức hệ thống dựa trên sơ đồ tổng thể và source code hiện tại của repository. Các chi tiết về mô hình dữ liệu, Rule Evaluation Engine, API, cache, kiểm thử và bảo mật được trình bày ở các mục chuyên sâu phía sau.

2.4.1 Các thành phần chính

Hình 2.2 mô tả hệ thống theo ba lớp chính: client/frontend, backend API và persistence/cache. Backend NestJS là ranh giới truy cập duy nhất cho cả luồng quản trị và luồng evaluation; Rule Evaluation Engine được tách khỏi controller để giữ logic quyết định xác định, dễ kiểm thử và không phụ thuộc giao diện.



Hình 2.2: Kiến trúc tổng thể của hệ thống Feature Flag Platform

Frontend / Client layer

Lớp client gồm `apps/admin`, `apps/demo` và `packages/js-sdk`. Admin Dashboard là React/Vite SPA cho project, flag, rule, group, audit log, history, RBAC demo và statistics. Demo Application mô phỏng ứng dụng thương mại điện tử nhỏ, gồm frontend storefront và một demo backend cùng service; frontend dùng kết quả evaluation để bật hoặc ẩn tính năng, còn demo backend dùng cùng SDK để guard các endpoint tính năng trước khi trả dữ liệu. `@ffp/js-sdk` chuẩn hóa cách gọi Evaluation API, validate response, xử lý timeout/lỗi mạng và trả fallback fail-closed cho cả luồng tích hợp phía browser và phía server demo.

Backend API và lớp bảo vệ

Backend nằm trong `apps/backend` và được triển khai bằng NestJS với global prefix `/v1`, DTO validation, Swagger, CORS theo admin/demo origin và request context middleware. `AuthModule` cung cấp demo authentication guard và permission guard cho control-plane endpoint; Health API và Evaluation API được đánh dấu public để phục vụ runtime evaluation.

Management modules

Các module quản trị gồm project, feature flag, flag rule, flag group, sample user, audit log/history và statistics. Mutation quan trọng đi qua validation, permission check, Prisma transaction, audit log append-only và cache invalidation. Cách tổ chức này giữ logic quản trị tập trung ở backend thay vì phân tán sang Dashboard.

Evaluation runtime

Data plane bắt đầu tại `POST /v1/evaluate`. `EvaluationService` lấy snapshot từ cache hoặc repository, gọi hàm thuần `evaluateFlag`, rồi trả kết quả gồm `projectKey`, `flagKey`, `enabled`, `variant`, `reason` và `matchedRuleId`. Engine áp dụng precedence: archived, config disabled, group kill switch, flag kill switch, global on, user allowlist, role targeting, percentage rollout và default off.

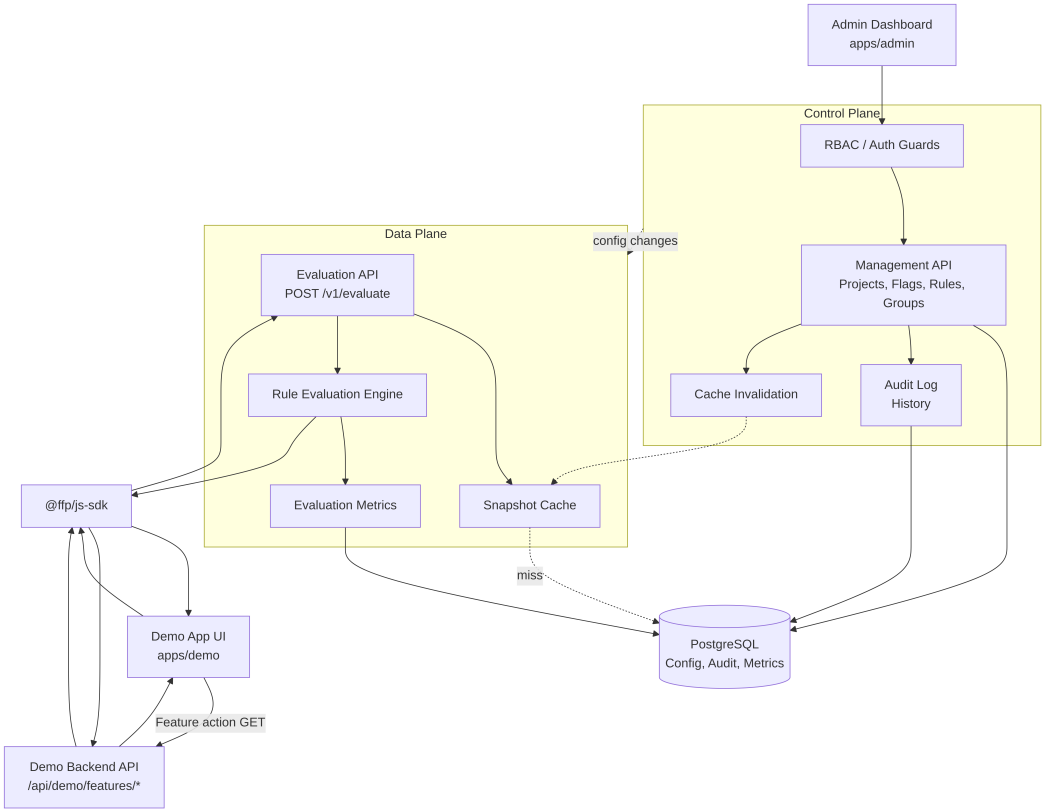
Persistence, cache và deployment

PostgreSQL là nguồn sự thật cho project, environment, flag, config, rule, group, audit log, sample user và aggregate metric. Evaluation Snapshot Cache chỉ lưu cấu hình cần cho evaluation, không lưu context người dùng hay kết quả theo user; provider có thể là `memory`, `none` hoặc optional `redis`. Repository hỗ trợ chạy bằng npm workspace hoặc Docker Compose với PostgreSQL, migration, seed, backend, admin, demo, demo staging và optional Redis. Riêng service demo chạy Node server để vừa phục vụ frontend đã build vừa expose `/api/demo/*` cho phần backend API minh họa.

2.4.2 Control plane và data plane

Kiến trúc tách *control plane* và *data plane* vì Feature Flag có hai loại tải khác nhau: thay đổi cấu hình ít hơn nhưng evaluation có thể xảy ra liên tục từ ứng dụng tích hợp [3, 5]. Control plane gồm Admin Dashboard, Auth/Permission Guards, Management API, Audit Log/History và

Cache Invalidation. Data plane gồm Evaluation API, Snapshot Cache, Rule Evaluation Engine và Evaluation Metrics.



Hình 2.3: Phân tách control plane và data plane trong hệ thống

Trong control plane, mọi mutation quan trọng được backend kiểm tra quyền, validate dữ liệu, ghi PostgreSQL và tạo audit entry với before/after snapshot. Trong data plane, ứng dụng khách chỉ gửi project key, environment key nếu có, flag key và context không nhạy cảm; Evaluation API đọc snapshot cấu hình, chạy engine và trả reason code. Demo backend trong `apps/demo` được xem như một ứng dụng tích hợp phía server: nó gọi data plane qua SDK trước khi trả dữ liệu tính năng cho browser. Cache và metric chỉ là tối ưu/quan sát best-effort nên không được làm thay đổi quyết định bật/tắt.

2.4.3 Luồng quản trị Feature Flag

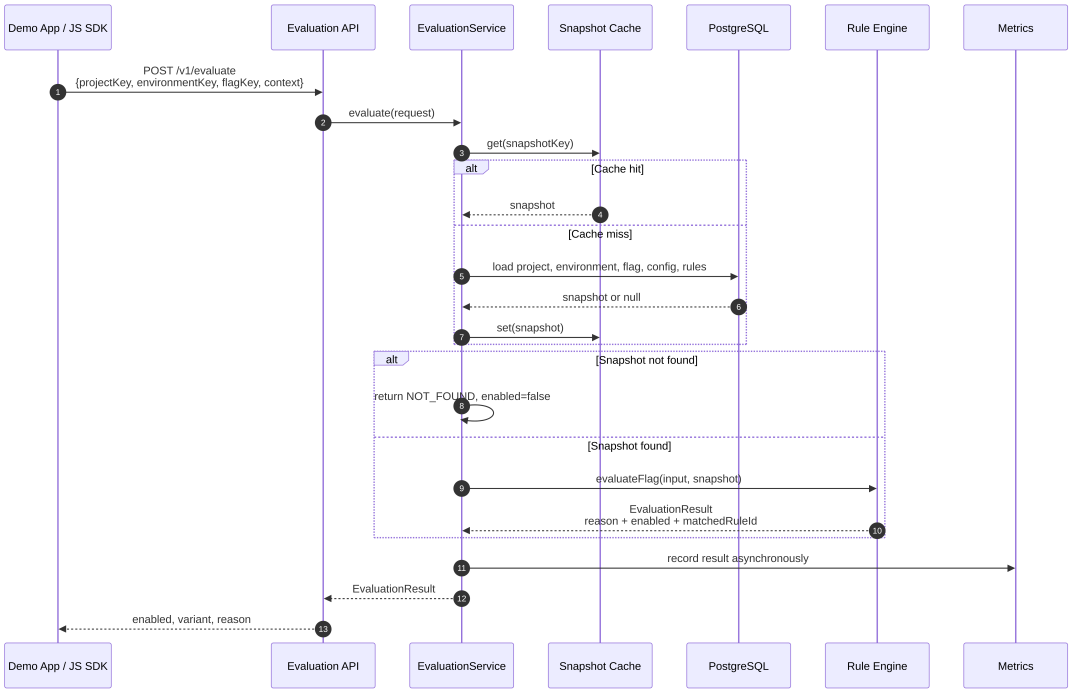
Người dùng quản trị chọn project trên Dashboard rồi tạo/sửa flag, đổi lifecycle, cấu hình status/serving mode theo environment, thay thế rule, gán group hoặc bật kill switch. Dashboard chỉ gửi request và hiển thị trạng thái; backend chịu trách nhiệm kiểm tra quyền, kiểm tra dữ liệu, lưu bằng Prisma transaction và ghi audit log. Rule được lưu dưới dạng `USER_ALLOWLIST`,

`ROLE_TARGETING` hoặc `PERCENTAGE_ROLLOUT` với `priority` và tham số `JSONB`.

Sau khi mutation commit, cache invalidator xóa snapshot liên quan để evaluation tiếp theo không dùng cấu hình cũ. Ví dụ, thay đổi flag config/rule sẽ invalidation snapshot của flag; gán group hoặc bật group kill switch sẽ invalidation các flag bị ảnh hưởng. Nhờ vậy, thay đổi từ Dashboard có thể được truy vết qua audit/history và phản ánh ở Demo Application mà không cần redeploy.

2.4.4 Luồng đánh giá Feature Flag

Luồng evaluation bắt đầu từ Demo Application hoặc SDK và kết thúc bằng một response có reason code. Evaluation API không thay đổi cấu hình và không ghi audit log quản trị. Hình 2.4 mô tả flow chính: nhận request, lấy snapshot từ cache hoặc PostgreSQL, gọi Rule Evaluation Engine, ghi metric best-effort và trả kết quả cho client.



Hình 2.4: Luồng đánh giá Feature Flag từ ứng dụng khách đến Rule Evaluation Engine

Nếu snapshot không tồn tại, service trả `enabled=false` với `reason=NOT_FOUND`; nếu có lỗi bất thường, hệ thống fail-closed với `reason=ERROR`. Nếu snapshot hợp lệ, engine xét các điều kiện terminal trước rồi mới xét global on và rule targeting. Percentage rollout dùng stable hashing trên project key, flag key và targeting key để kết quả lặp lại ổn định cho cùng một context.

2.4.5 Luồng tích hợp ứng dụng khách

Ứng dụng khách có thể gọi trực tiếp Evaluation API hoặc dùng `@ffp/js-sdk`. Trong project này, Demo Application dùng SDK để cấu hình `baseUrl`, `projectKey`, optional `environmentKey`, timeout và `fetch`; sau đó gọi `evaluate`, `isEnabled` hoặc `getVariant`. Frontend dùng SDK để quyết định UI, còn demo backend dùng SDK để guard `GET/api/demo/features/{featureKey}` trước khi trả dữ liệu tính năng. Context chỉ là đầu vào targeting, thường gồm `targetingKey`, `userId` mẫu, `roles` và `attributes`, không phải bằng chứng phân quyền.

Khi API timeout, lỗi mạng, HTTP lỗi, JSON sai shape hoặc response không khớp project/flag, SDK trả fallback an toàn: `enabled=false`, `variant=off`, `reason=ERROR` và `errorSource=CLIENT`. Quy tắc này ngăn lỗi hạ tầng vô tình mở tính năng chưa sẵn sàng.

2.4.6 Lựa chọn công nghệ

NestJS được chọn cho backend vì hỗ trợ module hóa, dependency injection, guard/pipe/filter, Swagger và Jest, phù hợp với yêu cầu tách controller, service, repository và domain evaluation. React/Vite được chọn cho Admin Dashboard và Demo Application vì nhẹ, dễ chia component và phù hợp npm workspace. PostgreSQL và Prisma được dùng để có transaction, khóa ngoại, unique constraint, `JSONB`, migration và data access có kiểu.

Cache snapshot dùng in-memory mặc định để đơn giản cho demo một instance; optional Redis thể hiện hướng mở rộng khi cần cache chia sẻ. JavaScript SDK được tách thành package riêng để ổn định contract tích hợp và đóng gói fail-closed behavior ở client. Docker Compose được chọn để khởi chạy lặp lại toàn bộ PostgreSQL, migration, seed, backend, admin và demo cho chấm điểm/trình diễn. Các phương án như tách microservice evaluation, message broker hoặc local-evaluation SDK phức tạp được để ngoài phạm vi vì làm tăng chi phí vận hành so với mục tiêu mini project.

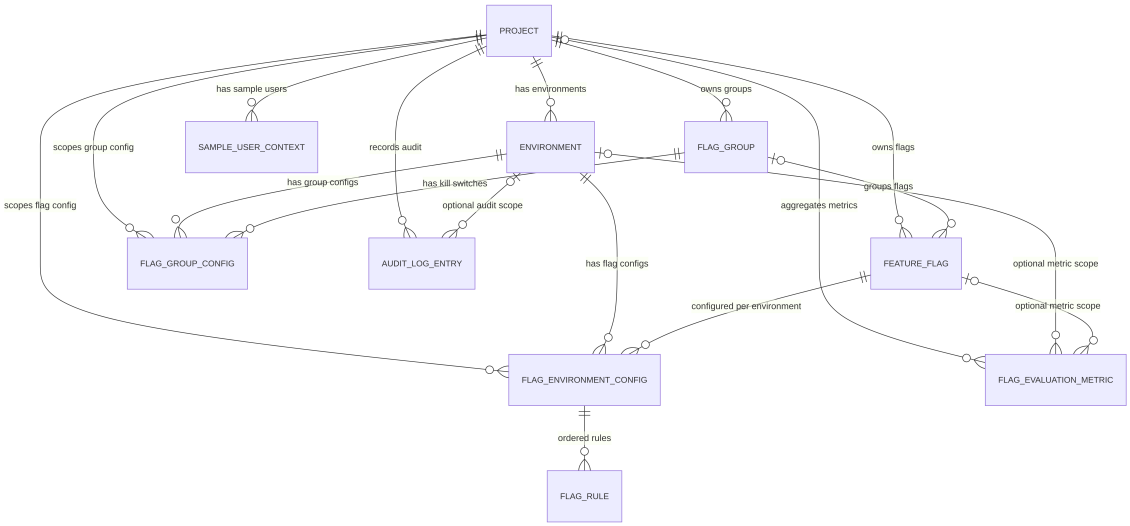
2.5 Thiết kế dữ liệu

Phần này mô tả mô hình dữ liệu đang được triển khai trong source code backend, chủ yếu dựa trên Prisma schema tại `apps/backend/prisma/schema.prisma` và các migration PostgreSQL tương ứng. Mô hình được thiết kế để lưu cấu hình Feature Flag, cấu hình theo environment, rule targeting, group kill switch, user context mẫu, audit log append-only và metric tổng hợp

phục vụ dashboard quan sát.

2.5.1 Mô hình dữ liệu tổng thể

Hình 2.5 là sơ đồ thực entity liên kết rút gọn của hệ thống Feature Flag Platform. Sơ đồ chỉ nhấn mạnh quan hệ chính giữa các bảng để người đọc dễ theo dõi luồng dữ liệu từ cấu hình quản trị đến runtime evaluation.



Hình 2.5: Sơ đồ thực thể liên kết rút gọn của hệ thống Feature Flag Platform

Trung tâm của mô hình là `projects`. Mỗi project sở hữu nhiều environment, feature flag, flag group, sample user context, audit log và metric evaluation. Bảng `feature_flags` lưu định nghĩa ổn định của flag, trong khi `flag_environment_configs` lưu trạng thái runtime của từng flag theo từng environment; nhờ đó cùng một flag có thể tắt ở production, bật thử ở staging hoặc chạy theo rule targeting mà không cần nhân bản định nghĩa flag. Bảng `flag_rules` phụ thuộc vào từng cấu hình flag–environment và lưu rule có thứ tự bằng trường `priority`. Bảng `flag_groups` cho phép gom các flag liên quan, còn `flag_group_configs` lưu group kill switch theo environment để ngắt nhanh cả nhóm nhưng không ảnh hưởng môi trường khác. `sample_user_contexts` là dữ liệu mẫu phục vụ demo và kiểm thử targeting; đây không phải user store bắt buộc của hệ thống production. `audit_log_entries` ghi lại mutation quản trị theo dạng append-only với actor, request ID và snapshot trước/sau. `flag_evaluation_metrics` chỉ lưu số liệu tổng hợp theo bucket thời gian, project key, environment key, flag key, reason và kết quả bật/tắt; bảng này không lưu context người dùng thô để giữ ranh giới riêng tư của data plane.

Bảng `projects`

Bảng 2.14: Mô tả dữ liệu bảng `projects`

Trường Prisma	Cột DB / kiểu	Ràng buộc / mặc định	Ý nghĩa
`id`	`id` / `TEXT`	Khóa chính, `@default(cuid())`	Định danh nội bộ của project.
`key`	`key` / `VARCHAR(64)`	Bắt buộc, duy nhất toàn hệ thống	Khóa nghiệp vụ dùng trong API, cache key và seed data.
`name`	`name` / `VARCHAR(120)`	Bắt buộc	Tên hiển thị của project.
`description`	`description` / `TEXT`	Có thể rỗng	Mô tả mục tiêu hoặc phạm vi project.
`createdAt`	`created_at` / `TIMESTAMP(3)`	Mặc định thời điểm tạo	Thời điểm tạo bản ghi.
`updatedAt`	`updated_at` / `TIMESTAMP(3)`	Prisma `@updatedAt`	Thời điểm cập nhật gần nhất.
`deletedAt`	`deleted_at` / `TIMESTAMP(3)`	Có thể rỗng, có index	Mốc soft delete của project.
`deletedBy`	`deleted_by` / `VARCHAR(120)`	Có thể rỗng	Actor thực hiện soft delete.

Bảng `environments`

Bảng 2.15: Mô tả dữ liệu bảng `environments`

Trường Prisma	Cột DB / kiểu	Ràng buộc / mặc định	Ý nghĩa
`id`	`id` / `TEXT`	Khóa chính, `@default(cuid())`	Định danh nội bộ của environment.
`projectId`	`project_id` / `TEXT`	FK tới `projects.id`, `ON DELETE CASCADE`	Project sở hữu environment.
`key`	`key` / `VARCHAR(64)`	Bắt buộc, duy nhất theo `project_id`	Khóa môi trường như `staging` hoặc `production`.
`name`	`name` / `VARCHAR(120)`	Bắt buộc	Tên hiển thị của environment.
`description`	`description` / `TEXT`	Có thể rỗng	Mô tả phạm vi triển khai.
`isDefault`	`is_default` / `BOOLEAN`	Mặc định `false`; tối đa một default trong mỗi project	Đánh dấu environment mặc định cho seed và cấu hình ban đầu.
`sortOrder`	`sort_order` / `INTEGER`	Mặc định `0`	Thứ tự hiển thị trong UI/API.
`createdAt`	`created_at` / `TIMESTAMP(3)`	Mặc định thời điểm tạo	Thời điểm tạo bản ghi.
`updatedAt`	`updated_at` / `TIMESTAMP(3)`	Prisma `@updatedAt`	Thời điểm cập nhật gần nhất.

Bảng feature_flags

Bảng 2.16: Mô tả dữ liệu bảng feature_flags

Trường Prisma	Cột DB / kiểu	Ràng buộc / mặc định	Ý nghĩa
id	id / TEXT	Khóa chính, @default(cuid())	Định danh nội bộ của flag.
projectId	project_id / TEXT	FK tới projects.id, ON DELETE RESTRICT	Project sở hữu flag.
groupId	group_id / TEXT	Có thể rỗng; FK ghép (project_id, group_id) tới flag_groups	Nhóm flag tùy chọn.
key	key / VARCHAR(64)	Bắt buộc, duy nhất theo project_id	Khóa nghiệp vụ của flag.
name	name / VARCHAR(120)	Bắt buộc	Tên hiển thị của flag.
description	description / TEXT	Có thể rỗng	Mô tả tính năng được kiểm soát.
lifecycleStatus	lifecycle_status / enum lifecycle	Mặc định ACTIVE; có index theo project	Trạng thái vòng đời ACTIVE hoặc ARCHIVED.
archivedAt	archived_at / TIMESTAMP(3)	Có thể rỗng	Thời điểm archive flag.
createdAt	created_at / TIMESTAMP(3)	Mặc định thời điểm tạo	Thời điểm tạo bản ghi.
updatedAt	updated_at / TIMESTAMP(3)	Prisma @updatedAt	Thời điểm cập nhật gần nhất.
deletedAt	deleted_at / TIMESTAMP(3)	Có thể rỗng, index theo project	Mốc soft delete của flag.
deletedBy	deleted_by / VARCHAR(120)	Có thể rỗng	Actor thực hiện soft delete.

Bảng flag_groups

Bảng 2.17: Mô tả dữ liệu bảng flag_groups

Trường Prisma	Cột DB / kiểu	Ràng buộc / mặc định	Ý nghĩa
id	id / TEXT	Khóa chính, @default(cuid())	Định danh nội bộ của group.
projectId	project_id / TEXT	FK tới projects.id, ON DELETE RESTRICT	Project sở hữu group.
key	key / VARCHAR(64)	Bắt buộc, duy nhất theo project_id	Khóa nghiệp vụ của group.
name	name / VARCHAR(120)	Bắt buộc	Tên hiển thị của group.
createdAt	created_at / TIMESTAMP(3)	Mặc định thời điểm tạo	Thời điểm tạo bản ghi.
updatedAt	updated_at / TIMESTAMP(3)	Prisma @updatedAt	Thời điểm cập nhật gần nhất.

Bảng `flag_group_configs`

Bảng 2.18: Mô tả dữ liệu bảng `flag_group_configs`

Trường Prisma	Cột DB / kiểu	Ràng buộc / mặc định	Ý nghĩa
`id`	`id` / `TEXT`	Khóa chính, `@default(cuid())`	Định danh nội bộ của cấu hình group.
`projectId`	`project_id` / `TEXT`	FK tới `projects.id`, `ON DELETE RESTRICT`	Project chứa group và environment.
`groupId`	`group_id` / `TEXT`	FK ghép tới `flag_groups(project_id,id)`, `ON DELETE CASCADE`	Group được cấu hình.
`environmentId`	`environment_id` / `TEXT`	FK ghép tới `environments(project_id,id)`, `ON DELETE RESTRICT`	Environment áp dụng cấu hình group.
`killSwitch`	`kill_switch` / `BOOLEAN`	Mặc định `false`; duy nhất theo `(group_id, environment_id)`	Trạng thái ngắt nhanh toàn bộ group trong environment.
`createdAt`	`created_at` / `TIMESTAMP(3)`	Mặc định thời điểm tạo	Thời điểm tạo bản ghi.
`updatedAt`	`updated_at` / `TIMESTAMP(3)`	Prisma `@updatedAt`	Thời điểm cập nhật gần nhất.

Bảng `flag_environment_configs`

Bảng 2.19: Mô tả dữ liệu bảng `flag_environment_configs`

Trường Prisma	Cột DB / kiểu	Ràng buộc / mặc định	Ý nghĩa
`id`	`id` / `TEXT`	Khóa chính, `@default(cuid())`	Định danh nội bộ của cấu hình flag.
`projectId`	`project_id` / `TEXT`	FK tới `projects.id`, `ON DELETE RESTRICT`	Project chứa flag và environment.
`flagId`	`flag_id` / `TEXT`	FK ghép tới `feature_flags(project_id,id)`, `ON DELETE CASCADE`	Flag được cấu hình.
`environmentId`	`environment_id` / `TEXT`	FK ghép tới `environments(project_id,id)`, `ON DELETE RESTRICT`	Environment áp dụng cấu hình flag.
`status`	`status` / enum `FlagConfigStatus`	Mặc định `DISABLED`; có index theo environment	Trạng thái cấu hình `ENABLED` hoặc `DISABLED`.
`servingMode`	`serving_mode` / enum `ServingMode`	Mặc định `TARGETED`	Chế độ `GLOBAL_ON` hoặc `TARGETED`.
`killSwitch`	`kill_switch` / `BOOLEAN`	Mặc định `false`	Công tắc ngắt nhanh riêng cho flag trong environment.
`createdAt`	`created_at` / `TIMESTAMP(3)`	Mặc định thời điểm tạo	Thời điểm tạo bản ghi.
`updatedAt`	`updated_at` / `TIMESTAMP(3)`	Prisma `@updatedAt`	Thời điểm cập nhật gần nhất.

Bảng `flag_rules`

Bảng 2.20: Mô tả dữ liệu bảng `flag_rules`

Trường Prisma	Cột DB / kiểu	Ràng buộc / mặc định	Ý nghĩa
`id`	`id` / `TEXT`	Khóa chính, `@default(cuid())`	Định danh nội bộ của rule.
`flagConfigId`	`flag_config_id` / `TEXT`	FK tới `flag_environment_configs.id`, `ON DELETE CASCADE`	Cấu hình flag–environment chứa rule.
`type`	`type` / enum `RuleType`	Bắt buộc	Loại rule: `USER_ALLOWLIST`, `ROLE_TARGETING` hoặc `PERCENTAGE_ROLLOUT`.
`priority`	`priority` / `INTEGER`	Bắt buộc, duy nhất theo `flag_config_id`; validation `>= 0`	Thứ tự xét rule trong cùng cấu hình.
`enabled`	`enabled` / `BOOLEAN`	Mặc định `true`	Cho phép bỏ qua một rule mà không xóa dữ liệu.
`parameters`	`parameters` / `JSONB`	Bắt buộc; validate theo `type` ở back-end	Tham số rule như danh sách user, role hoặc phần trăm rollout.
`createdAt`	`created_at` / `TIMESTAMP(3)`	Mặc định thời điểm tạo	Thời điểm tạo bản ghi.
`updatedAt`	`updated_at` / `TIMESTAMP(3)`	Prisma `@updatedAt`	Thời điểm cập nhật gần nhất.

Bảng `flag_evaluation_metrics`

Bảng 2.21: Mô tả dữ liệu bảng `flag_evaluation_metrics`

Trường Prisma	Cột DB / kiểu	Ràng buộc / mặc định	Ý nghĩa
`id`	`id` / `TEXT`	Khóa chính, `@default(cuid())`	Định danh nội bộ của bucket metric.
`projectId`	`project_id` / `TEXT`	Có thể rỗng; FK tới `projects.id`, `ON DELETE SET NULL`	Liên kết tùy chọn tới project hiện còn tồn tại.
`environmentId`	`environment_id` / `TEXT`	Có thể rỗng; FK tới `environments.id`, `ON DELETE SET NULL`	Liên kết tùy chọn tới environment hiện còn tồn tại.
`flagId`	`flag_id` / `TEXT`	Có thể rỗng; FK tới `feature_flags.id`, `ON DELETE SET NULL`	Liên kết tùy chọn tới flag hiện còn tồn tại.
`projectKey`	`project_key` / `VARCHAR(64)`	Bắt buộc; thuộc unique bucket	Khóa project được giữ lại cho thống kê lịch sử.
`environmentKey`	`environment_key` / `VARCHAR(64)`	Bắt buộc; thuộc unique bucket	Khóa environment được giữ lại cho thống kê lịch sử.
`flagKey`	`flag_key` / `VARCHAR(64)`	Bắt buộc; thuộc unique bucket	Khóa flag được giữ lại cho thống kê lịch sử.
`bucketStart`	`bucket_start` / `TIMESTAMPTZ(3)`	Bắt buộc; thuộc unique bucket	Thời điểm bắt đầu bucket tổng hợp.
`reason`	`reason` / `VARCHAR(40)`	Bắt buộc; thuộc unique bucket	Reason code của kết quả evaluation.
`enabled`	`enabled` / `BOOLEAN`	Bắt buộc; thuộc unique bucket	Kết quả bật/tắt của evaluation.
`count`	`count` / `INTEGER`	Mặc định `1`	Số lượt evaluation được cộng dồn trong bucket.
`createdAt`	`created_at` / `TIMESTAMP(3)`	Mặc định thời điểm tạo	Thời điểm tạo bucket.
`updatedAt`	`updated_at` / `TIMESTAMP(3)`	Prisma `@updatedAt`	Thời điểm cập nhật bucket gần nhất.

Bảng `sample_user_contexts`

Bảng 2.22: Mô tả dữ liệu bảng `sample_user_contexts`

Trường Prisma	Cột DB / kiểu	Ràng buộc / mặc định	Ý nghĩa
`id`	`id` / `TEXT`	Khóa chính, `@default(cuid())`	Định danh nội bộ của user context mẫu.
`projectId`	`project_id` / `TEXT`	FK tới `projects.id`, `ON DELETE CASCADE`	Project chứa context mẫu.
`displayName`	`display_name` / `VARCHAR(120)`	Bắt buộc	Tên hiển thị trong dashboard/demo.
`targetingKey`	`targeting_key` / `VARCHAR(120)`	Bắt buộc, duy nhất theo `project_id`	Khóa ổn định, không chứa PII, dùng cho percentage rollout.
`userId`	`user_id` / `VARCHAR(120)`	Có thể rỗng	User ID mẫu dùng cho user allowlist.
`roles`	`roles` / `JSONB`	Mặc định `[]`	Danh sách role mẫu.
`attributes`	`attributes` / `JSONB`	Mặc định `{}`	Thuộc tính mở rộng phục vụ demo.
`createdAt`	`created_at` / `TIMESTAMP(3)`	Mặc định thời điểm tạo	Thời điểm tạo bản ghi.
`updatedAt`	`updated_at` / `TIMESTAMP(3)`	Prisma `@updatedAt`	Thời điểm cập nhật gần nhất.

Bảng `audit_log_entries`

Bảng 2.23: Mô tả dữ liệu bảng `audit_log_entries`

Trường Prisma	Cột DB / kiểu	Ràng buộc / mặc định	Ý nghĩa
`id`	`id` / `TEXT`	Khóa chính, `@default(cuid())`	Định danh nội bộ của audit entry.
`projectId`	`project_id` / `TEXT`	FK tới `projects.id`, `ON DELETE RESTRICT`	Project phát sinh thay đổi.
`projectKey`	`project_key` / `VARCHAR(64)`	Bắt buộc, có index theo thời gian	Khóa project được lưu kèm để truy vết ổn định.
`environmentId`	`environment_id` / `TEXT`	Có thể rỗng; FK tới `environments.id`, `ON DELETE SET NULL`	Environment liên quan nếu mutation có phạm vi môi trường.
`environmentKey`	`environment_key` / `VARCHAR(64)`	Có thể rỗng, có index theo project và thời gian	Khóa environment được lưu kèm để truy vết.
`targetType`	`target_type` / enum `AuditTargetType`	Bắt buộc	Loại đối tượng bị thay đổi.
`targetId`	`target_id` / `TEXT`	Bắt buộc	ID đối tượng bị thay đổi.
`targetKey`	`target_key` / `VARCHAR(120)`	Có thể rỗng, có index theo target	Khóa nghiệp vụ của đối tượng bị thay đổi.
`action`	`action` / enum `AuditAction`	Bắt buộc, có index theo project và thời gian	Hành động quản trị đã xảy ra.
`actor`	`actor` / `VARCHAR(120)`	Bắt buộc, có index theo project và thời gian	Danh tính actor do backend resolve.
`before`	`before` / `JSONB`	Có thể rỗng	Snapshot trước khi thay đổi.
`after`	`after` / `JSONB`	Có thể rỗng	Snapshot sau khi thay đổi.
`metadata`	`metadata` / `JSONB`	Có thể rỗng	Dữ liệu bổ sung như lý do hoặc phạm vi tác động.
`requestId`	`request_id` / `VARCHAR(120)`	Bắt buộc	ID request để tương quan log.
`createdAt`	`created_at` / `TIMESTAMP(3)`	Mặc định thời điểm tạo; có trigger cấm update/delete	Thời điểm ghi audit entry.

2.5.2 Chuẩn hóa dữ liệu

Mô hình dữ liệu của hệ thống được thiết kế chủ yếu theo các dạng chuẩn cơ bản của cơ sở dữ liệu quan hệ. Ở mức 1NF, mỗi bảng có khóa chính riêng và mỗi cột lưu một loại giá trị xác định; các quan hệ nhiều giá trị như nhiều rule của một cấu hình flag không được nhồi vào bảng flag mà được tách sang `FlagRule`. Một số trường linh hoạt như `parameters`, `roles`, `attributes` hoặc snapshot audit dùng `JSONB`; đây là ngoại lệ có kiểm soát để biểu diễn dữ liệu bán cấu trúc, không dùng để thay thế các quan hệ lõi giữa project, environment, flag, group và rule.

Ở mức 2NF và 3NF, các thuộc tính phụ thuộc vào đúng thực thể sở hữu chúng. Thông tin định danh của flag như key, tên, mô tả và lifecycle nằm trong `FeatureFlag`; trạng thái runtime theo từng môi trường nằm trong `FlagEnvironmentConfig`; group kill switch theo môi trường nằm trong `FlagGroupConfig`; rule targeting nằm trong `FlagRule`. Cách tách này loại bỏ phụ thuộc bộ phận và phụ thuộc bắc cầu phổ biến, ví dụ không lặp lại tên flag trong từng environment

config và không lưu trạng thái production/staging trực tiếp trong bảng `FeatureFlag`.

Một số dữ liệu được phi chuẩn hóa có chủ đích để phục vụ vận hành. `AuditLogEntry` lưu thêm các key và snapshot trước/sau để lịch sử vẫn đọc được khi bản ghi nghiệp vụ thay đổi; `FlagEvaluationMetric` lưu key dạng chuỗi bên cạnh khóa ngoại tùy chọn để thống kê lịch sử không mất ý nghĩa khi bản ghi gốc bị xóa mềm hoặc không còn liên kết. Những điểm này không làm thay đổi nguồn sự thật của cấu hình Feature Flag, mà chỉ hỗ trợ audit, history và dashboard thống kê.

2.5.3 Ràng buộc toàn vẹn dữ liệu

Ràng buộc dữ liệu được đặt ở cả Prisma/PostgreSQL và tầng validation của backend để cấu hình Feature Flag hợp lệ trước khi đi vào Rule Evaluation Engine. Nội dung dưới đây chỉ tổng hợp các ràng buộc toàn vẹn chính; mô tả trường dữ liệu chi tiết của từng bảng đã được trình bày ở mục 2.5.1.

Ràng buộc khóa và tính duy nhất

Bảng 2.24: Các ràng buộc duy nhất chính trong mô hình dữ liệu

Thực thể	Ràng buộc
`Project`	`key` duy nhất toàn hệ thống.
`Environment`	`projectId, key` duy nhất trong project; mỗi project có tối đa một environment mặc định.
`FeatureFlag`	`projectId, key` duy nhất trong project.
`FlagGroup`	`projectId, key` duy nhất trong project.
`FlagEnvironmentConfig`	`flagId, environmentId` duy nhất để một flag chỉ có một cấu hình trong một environment.
`FlagGroupConfig`	`groupId, environmentId` duy nhất để một group chỉ có một kill switch trong một environment.
`FlagRule`	`flagConfigId, priority` duy nhất để thứ tự rule trong một cấu hình là xác định.
`SampleUserContext`	`projectId, targetingKey` duy nhất trong project.
`FlagEvaluationMetric`	Một bucket metric duy nhất theo project key, environment key, flag key, thời điểm bucket, reason và kết quả bật/tắt.

Ngoài các khóa nghiệp vụ trên, schema còn có các unique constraint ghép `projectId, id` cho `Environment`, `FeatureFlag` và `FlagGroup`. Các cặp này được dùng làm đích của khóa ngoại ghép, giúp ngăn trường hợp một config vô tình nối flag, group hoặc environment thuộc project khác.

Ràng buộc quan hệ tham chiếu

Khóa ngoại bảo đảm các bảng con không trỏ tới project, environment, flag, group hoặc config không tồn tại. Các quan hệ cấu hình quan trọng dùng khóa ngoại ghép theo `projectId`: `FlagEnvironmentConfig` nối tới `FeatureFlag` và `Environment` trong cùng project; `FlagGroupConfig` nối tới `FlagGroup` và `Environment` trong cùng project; `FeatureFlag.groupId` cũng tham chiếu group theo cùng project. Nhờ đó, dữ liệu runtime không thể ghép lẫn cấu hình giữa hai project khác nhau.

Hành vi xóa được chọn theo vòng đời dữ liệu. `FlagRule` bị xóa cascade theo `FlagEnvironmentConfig`; `FlagEnvironmentConfig` cascade theo `FeatureFlag`; `FlagGroupConfig` cascade theo `FlagGroup`; `SampleUserContext` cascade theo `Project`. Ngược lại, các quan hệ cần bảo toàn lịch sử như project–feature flag, project–flag group, project–flag config và project–audit log dùng `Restrict`. Bảng `AuditLogEntry` và `FlagEvaluationMetric` dùng `SetNull` cho một số liên kết environment/metric để vẫn giữ được bản ghi lịch sử hoặc số liệu tổng hợp khi bản ghi gốc không còn tồn tại.

Ràng buộc giá trị và tham số rule

Các trường trạng thái dùng enum thay vì chuỗi tự do: lifecycle của flag chỉ là `ACTIVE` hoặc `ARCHIVED`; trạng thái config chỉ là `ENABLED` hoặc `DISABLED`; serving mode chỉ là `GLOBAL_ON` hoặc `TARGETED`; rule type chỉ thuộc ba loại `USER_ALLOWLIST`, `ROLE_TARGETING` và `PERCENTAGE_ROLLOUT`. Các key nghiệp vụ được backend validate theo định dạng slug chữ thường dài 3–64 ký tự, chỉ gồm chữ cái, chữ số và dấu gạch ngang.

Với `FlagRule.parameters`, database lưu bằng `JSONB` còn backend kiểm tra shape theo `type`. Rule `USER_ALLOWLIST` cần danh sách `userIds` không rỗng; `ROLE_TARGETING` cần danh sách `roles` không rỗng; `PERCENTAGE_ROLLOUT` cần `percentage` là số từ 0 đến 100 và có tối đa hai chữ số thập phân. Tầng service cũng kiểm tra trùng `priority` khi thay thế rule để nhất quán với unique constraint trong database.

Ràng buộc append-only cho audit log

Audit log là nguồn truy vết cấu hình nên không được sửa hoặc xóa sau khi ghi. Backend không cung cấp luồng update/delete cho `AuditLogEntry`; đồng thời migration PostgreSQL đã tạo trigger `audit_log_entries_no_update` và `audit_log_entries_no_delete` để chặn trực tiếp

thao tác `UPDATE` và `DELETE` trên bảng audit. Các mutation quản trị quan trọng ghi audit entry trong cùng transaction với thay đổi cấu hình, giúp snapshot trước/sau phản ánh đúng trạng thái đã commit.

Chiến lược archive và soft delete

Flag không còn phục vụ runtime được chuyển sang lifecycle `ARCHIVED` để giữ lại định nghĩa, cấu hình và lịch sử thay đổi. Ngoài archive, hệ thống hiện có soft delete cho `Project` và `FeatureFlag` thông qua `deletedAt` và `deletedBy`; bản ghi bị soft delete được ẩn khỏi danh sách thông thường nhưng vẫn còn trong database để phục vụ khôi phục, audit/history và tránh phá vỡ các ràng buộc tham chiếu.

2.6 Thiết kế Rule Evaluation Engine

Rule Evaluation Engine là lõi quyết định của data plane: với một request runtime và một snapshot cấu hình đã chuẩn hóa, engine trả về flag đang *on/off*, variant, reason code và rule được match nếu có. Phần này được viết dựa trên source code backend hiện tại: các file engine, type, stable hash, service và repository trong module evaluation. Mục tiêu thiết kế là giữ kết quả xác định, mặc định an toàn, tách engine khỏi controller/ORM, đồng thời đủ rõ để dashboard, demo app và `@ffp/js-sdk` cùng hiểu một contract thống nhất.

2.6.1 Contract đầu vào và evaluation context

Evaluation API nhận khóa cấu hình gồm `projectKey`, `environmentKey` tùy chọn, `flagKey` và một `context` do ứng dụng khách gửi lên. `environmentKey` dùng để chọn cấu hình theo môi trường; nếu bỏ trống, backend dùng environment mặc định của project. Context chỉ phục vụ targeting/rollout, không thay thế xác thực người dùng cuối hay phân quyền nghiệp vụ của ứng dụng tích hợp.

Đoạn mã 2.2: Cấu trúc evaluation request

```
{
  "projectKey": "demo-project",
  "environmentKey": "production",
  "flagKey": "sample-feature",
  "context": {
```

```
    "targetingKey": "demo-user-regular",
    "userId": "demo-user-regular",
    "roles": ["beta-tester"],
    "attributes": {
      "plan": "pro",
      "country": "VN"
    }
  }
}
```

Trong contract này, `targetingKey` là khóa ổn định, không chứa PII, dùng cho percentage rollout; `userId` dùng cho user allowlist; `roles` dùng cho role targeting; `attributes` là vùng mở rộng cho các rule tương lai. Backend validate định dạng key và kiểu dữ liệu bằng DTO trước khi vào service. Request sai schema bị chặn ở biên API, còn request hợp lệ nhưng thiếu dữ liệu cho một loại rule sẽ được engine xử lý an toàn: allowlist hoặc role rule bị bỏ qua khi thiếu `userId`/`roles`; riêng percentage rollout trả `INVALID_CONTEXT` nếu đã đến lượt xét percentage rule nhưng `targetingKey` rỗng hoặc chỉ có khoảng trắng.

2.6.2 Snapshot cấu hình và luồng thực thi data plane

Engine không truy cập database trực tiếp. `EvaluationService` tạo `EvaluationInput`, thử đọc snapshot từ cache bằng `projectKey`, `environmentKey` và `flagKey`; nếu cache miss thì `EvaluationRepository` nạp dữ liệu từ PostgreSQL qua Prisma. Snapshot gồm lifecycle của flag, trạng thái config theo environment, serving mode, flag kill switch, group kill switch nếu flag thuộc group, danh sách rule đã có `id`, `type`, `priority`, `enabled` và `parameters`. Nếu project, environment, flag hoặc config không tồn tại, service không trả lỗi `404` cho data plane mà trả `enabled=false` với reason `NOT_FOUND`. Nếu cache lỗi, service fallback về repository; nếu repository hoặc engine lỗi bất ngờ, kết quả là `ERROR` theo nguyên tắc fail-closed. Việc ghi metric được thực hiện an toàn sau khi có kết quả và không được phép làm thay đổi response evaluation.

Cách tách snapshot khỏi engine tạo ba lợi ích. Thứ nhất, cùng một snapshot và cùng một context luôn tạo ra cùng một kết quả, thuận lợi cho unit test. Thứ hai, controller/service có thể thay đổi cache hoặc repository mà không làm nhiễu logic quyết định. Thứ ba, audit log và history vẫn thuộc control plane, không bị đưa vào runtime snapshot, giúp đường evaluate gọn

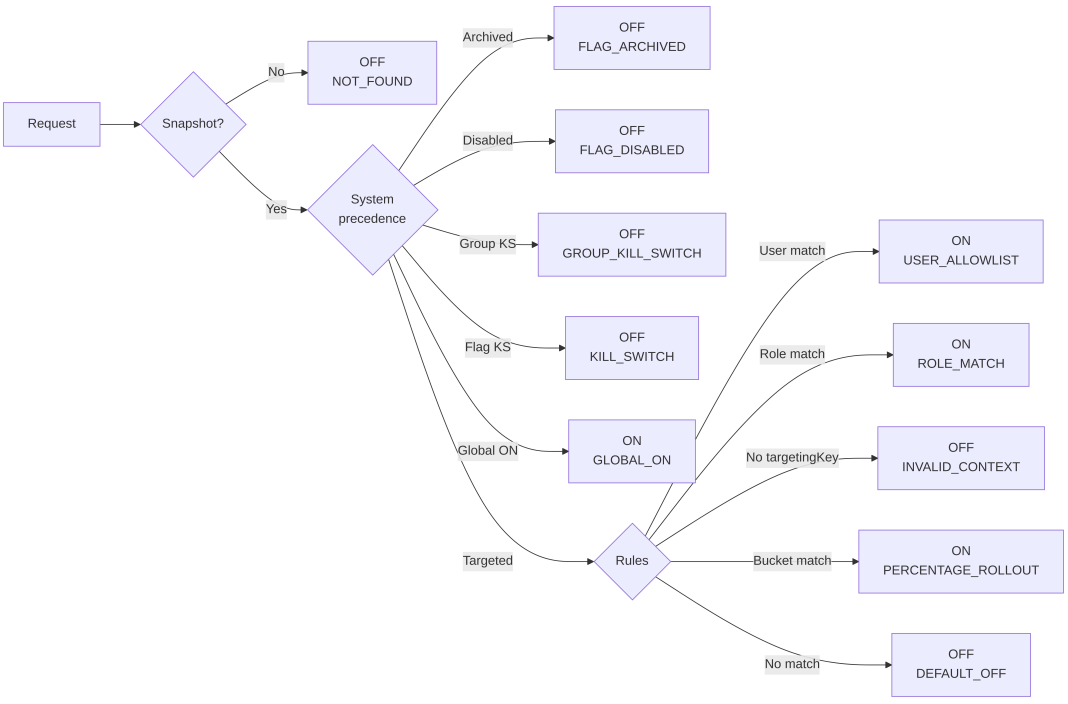
và không phơi bày cấu hình quản trị cho client.

2.6.3 Thứ tự ưu tiên khi đánh giá

Thứ tự ưu tiên là hợp đồng quan trọng nhất của Rule Evaluation Engine. Các điều kiện an toàn và kill switch luôn được xét trước rollout rule để không có rule nào vô tình bật lại một flag đã bị archive, disabled hoặc đang bị ngắt khẩn cấp. Hình 2.6 giữ nguyên sơ đồ flowchart của engine; các nhánh trong hình tương ứng trực tiếp với thứ tự trong source code `evaluateFlag`.

Đoạn mã 2.3: Thứ tự ưu tiên của Rule Evaluation Engine

```
FLAG_ARCHIVED
FLAG_DISABLED
GROUP_KILL_SWITCH
KILL_SWITCH
GLOBAL_ON
USER_ALLOWLIST
ROLE_MATCH
PERCENTAGE_ROLLOUT
DEFAULT_OFF
```



Hình 2.6: Flowchart thứ tự ưu tiên của Rule Evaluation Engine

Bảng 2.25: Tương tác giữa trạng thái và serving mode

Lifecycle/config	Kill switch	Serving mode	Kết quả ưu tiên
Archived	Bất kỳ	Bất kỳ	Tắt, `FLAG_ARCHIVED`
Disabled	Bất kỳ	Bất kỳ	Tắt, `FLAG_DISABLED`
Enabled	Group on	Bất kỳ	Tắt, `GROUP_KILL_SWITCH`
Enabled	Flag on	Bất kỳ	Tắt, `KILL_SWITCH`
Enabled	Off	Global on	Bật, `GLOBAL_ON`
Enabled	Off	Targeted	Xét rule theo precedence

Các nhánh hệ thống có ý nghĩa như sau. `FLAG_ARCHIVED` đứng đầu vì flag đã archive không còn được phục vụ runtime. `FLAG_DISABLED` phản ánh config theo environment đang tắt, nên mọi rule phía sau đều bị bỏ qua. `GROUP_KILL_SWITCH` cho phép tắt nhanh cả nhóm flag trong một environment sau khi đã qua kiểm tra archived/disabled. `KILL_SWITCH` là công tắc khẩn cấp riêng của flag. `GLOBAL_ON` bật flag cho mọi context khi flag đang enabled và serving mode là global on. Chỉ khi tất cả nhánh này không kết thúc evaluation, engine mới xét rule targeting theo thứ tự allowlist, role, percentage và cuối cùng là `DEFAULT_OFF`.

2.6.4 Rule targeting và stable percentage rollout

Rule runtime hiện có ba loại đúng với enum `RuleType`: `USER_ALLOWLIST`, `ROLE_TARGETING` và `PERCENTAGE_ROLLOUT`. Engine lọc các rule `enabled=true`, sắp xếp theo `priority` tăng dần, rồi xét theo type precedence cố định. Trong từng loại, rule nào match trước sẽ trả kết quả ngay kèm `matchedRuleId`. User allowlist so khớp chính xác `context.userId` với mảng `parameters.userIds`; role targeting match khi ít nhất một role trong `context.roles` nằm trong `parameters.roles`. Engine cố tình tách `userId` và `targetingKey`: `userId` là định danh nghiệp vụ cho allowlist, còn `targetingKey` là khóa rollout ổn định và nên không chứa PII.

Percentage rollout dùng stable hashing để chia `targetingKey` vào bucket cố định trong khoảng từ `0.00` đến dưới `100`. Source code hiện băm tổ hợp khóa project, khóa flag và targeting key đã trim; không dùng thời gian, số ngẫu nhiên hoặc trạng thái request. Vì vậy một người dùng giữ quyết định ổn định cho cùng project/flag. Digest SHA-256 được đọc 64 bit đầu tiên, lấy modulo `10000` rồi chia `100` để hỗ trợ percentage có hai chữ số thập phân. Rule hợp lệ khi `percentage` là số hữu hạn từ `0` đến `100` và tối đa hai chữ số thập phân; engine bật flag khi bucket nhỏ hơn percentage.

Đoạn mã 2.4: Tạo giá trị băm ổn định bằng SHA-256

```
const normalizedTargetingKey = targetingKey.trim();
const hashInput =
  `${projectKey}:${flagKey}:${normalizedTargetingKey}`;
const digest = createHash('sha256')
  .update(hashInput, 'utf8')
  .digest();
```

Đoạn mã 2.5: Chuyển đổi SHA-256 digest thành bucket phần trăm

```
const first64Bits = digest.readBigUInt64BE(0);
const bucketPercentage = Number(first64Bits % 10000n) / 100;
const enabled = bucketPercentage < percentage;
```

Thiết kế này cho phép rollout tăng dần theo tỷ lệ mà không làm người dùng "nhảy" giữa on/off sau mỗi request. Nếu targeting key thay đổi, bucket cũng có thể thay đổi, nên demo app và SDK cần chọn khóa rollout ổn định ngay từ đầu. Rule sai tham số bị bỏ qua thay vì bật tính năng ngoài ý muốn; đây là một phần của chiến lược safe default.

2.6.5 EvaluationResult, reason code và fallback

Response của data plane được giữ nhỏ và ổn định: client chỉ cần `enabled`/`variant` để quyết định hành vi, còn dashboard/demo/debug dùng `reason` và `matchedRuleId` để giải thích vì sao có kết quả đó. Response không trả toàn bộ rule parameters, danh sách user hay cấu hình quản trị nội bộ.

Đoạn mã 2.6: Cấu trúc EvaluationResult

```
{
  "projectKey": "demo-project",
  "flagKey": "sample-feature",
  "enabled": true,
  "variant": "on",
  "reason": "USER_ALLOWLIST",
  "matchedRuleId": "rule_123"
}
```

Đoạn mã 2.7: Tập reason code của Evaluation API

```
GLOBAL_ON

FLAG_DISABLED

FLAG_ARCHIVED

GROUP_KILL_SWITCH

KILL_SWITCH

USER_ALLOWLIST

ROLE_MATCH

PERCENTAGE_ROLLOUT

DEFAULT_OFF

NOT_FOUND

INVALID_CONTEXT

ERROR
```

Bảng 2.26: Reason code cho trạng thái hệ thống

Reason	**Ý nghĩa**
NOT_FOUND	Không tìm thấy project, environment, flag hoặc config cần evaluate.
FLAG_ARCHIVED	Flag không còn phục vụ runtime.
FLAG_DISABLED	Config theo environment đang disabled.
GROUP_KILL_SWITCH	Group kill switch đang bật cho environment hiện tại.
KILL_SWITCH	Kill switch riêng của flag đang bật.
GLOBAL_ON	Flag được bật toàn cục trong environment hiện tại.
ERROR	Lỗi không mong muốn, backend trả fail-closed.

Bảng 2.27: Quan hệ giữa mã lý do và matched rule

Reason	**Matched rule hoặc tình huống**
USER_ALLOWLIST	Rule allowlist match `context.userId`.
ROLE_MATCH	Rule role targeting match ít nhất một role của context.
PERCENTAGE_ROLLOUT	Bucket stable hashing nằm trong tỷ lệ rollout.
INVALID_CONTEXT	Đến percentage rollout nhưng thiếu `targetingKey` hợp lệ.
DEFAULT_OFF	Không có điều kiện hệ thống hoặc rule nào bật flag.

Nguyên tắc fallback chung là fail-closed: khi không chắc có nên bật tính năng hay không, hệ

thống trả tắt. Vì vậy project/flag/config không tồn tại trả `NOT_FOUND`; lỗi bất ngờ trả `ERROR`; rule không đủ tham số hoặc không đủ context không được phép tạo kết quả bật ngoài ý muốn. Dây là điểm quan trọng của feature flag trong release management: sự cố ở control plane, cache hoặc rule malformed không được biến thành phát hành ngoài kế hoạch.

Đoạn mã 2.8: Kết quả fallback mặc định

```
{
  "enabled": false,
  "variant": "off",
  "reason": "ERROR"
}
```

Đoạn mã 2.9: Xử lý lỗi nội bộ của Evaluation Engine

```
try {
  return evaluateFlag(input, snapshot);
} catch (error) {
  // enabled=false, variant="off", reason="ERROR"
  return errorResult(input);
}
```

2.6.6 Khả năng mở rộng và kiểm chứng

Engine được thiết kế để mở rộng bằng cách thêm `RuleType`, hàm evaluate tương ứng và reason code mới mà không phá vỡ response shape hiện tại. Những hướng phù hợp cho giai đoạn sau gồm targeting theo thuộc tính, organization, thời gian, vị trí hoặc multi-variant/A-B testing. Tuy nhiên mọi mở rộng phải giữ các ràng buộc cốt lõi: precedence hệ thống đứng trước targeting rule, percentage rollout xác định bằng stable hashing, response không lộ PII/cấu hình nhạy cảm, cache miss không đổi kết quả, metric lỗi không ảnh hưởng evaluation và default luôn là off.

Các unit test hiện khóa những hành vi quan trọng: `FLAG_ARCHIVED` đứng trước `FLAG_DISABLED`, `FLAG_DISABLED` đứng trước `GROUP_KILL_SWITCH`, `GROUP_KILL_SWITCH` đứng trước `KILL_SWITCH`, `KILL_SWITCH` đứng trước `GLOBAL_ON`; rule disabled bị bỏ qua; allowlist, role và percentage trả đúng `matchedRuleId`; targeting key rỗng trả `INVALID_CONTEXT`; stable hash trả bucket lặp lại

ổn định trong khoảng `0.00-99.99`. Nhờ đó phần engine vừa là nền tảng kỹ thuật của demo, vừa là bằng chứng thiết kế cho các yêu cầu recommended như cache snapshot, statistics dashboard, group kill switch và JavaScript SDK.

2.7 Thiết kế API

API của hệ thống được thiết kế theo REST, dùng JSON, đặt dưới tiền tố `/v1` và dùng quy ước `camelCase`. Backend NestJS sinh Swagger/OpenAPI từ controller và DTO; khi chạy, Swagger UI có tại `/docs` và đặc tả JSON có tại `/docs/json`. Thiết kế API tách *control plane* cho quản trị cấu hình, *data plane* cho runtime evaluation và *monitoring* cho audit/statistics; các endpoint dùng khóa nghiệp vụ ổn định như `projectKey`, `flagKey`, `groupKey` và request ID `X-Request-Id` để hỗ trợ truy vết.

2.7.1 Control plane API

Control plane quản lý project, flag, rule, group và sample user. Nhóm endpoint này yêu cầu bearer token demo `demoBearer`; backend resolve actor theo token, kiểm tra quyền theo vai trò `ADMIN`/`DEVELOPER`/`VIEWER`, chuẩn hóa pagination bằng `items`/`page`, và ghi audit log cho các mutation quan trọng. Environment chưa có CRUD riêng; dữ liệu environment được dùng qua seed/model và được tham chiếu bằng `environmentKey` trong evaluation, group config và statistics.

Project API

Bảng 2.28: Đặc tả Project API

Method	Endpoint	Quyền	Mục đích và dữ liệu chính
GET	`/v1/projects`	`CONTROL_PLANE_READ`	Liệt kê project, trả `items` và `page`. Query kế thừa pagination.
POST	`/v1/projects`	`PROJECT_MANAGE`	Tạo project với `key`, `name`, `description`. Trả `201 Created`.
GET	`/v1/projects/{projectKey}`	`CONTROL_PLANE_READ`	Xem chi tiết project theo khóa nghiệp vụ.
PATCH	`/v1/projects/{projectKey}`	`PROJECT_MANAGE`	Cập nhật `name` hoặc `description`; `key` là bất biến.
DELETE	`/v1/projects/{projectKey}`	`PROJECT_MANAGE`	Xóa project ở mức quản trị, trả `204 No Content`.

Feature Flag API

Bảng 2.29: Đặc tả Feature Flag API

Method	Endpoint	Quyền	Mục đích và dữ liệu chính
GET	`/v1/projects/{projectKey}/flags`	`CONTROL_PLANE_READ`	Liệt kê flag chưa soft-delete. Query hỗ trợ `search`, `status`, `lifecycleStatus` và pagination.
GET	`/v1/projects/{projectKey}/flags/deleted`	`CONTROL_PLANE_READ`	Liệt kê flag đã soft-delete để có thể khôi phục khi cần.
POST	`/v1/projects/{projectKey}/flags`	`FLAG_MANAGE`	Tạo flag với `key`, `name`, `description`; backend tạo config mặc định cho environment mặc định.
GET	`/v1/projects/{projectKey}/flags/{flagKey}`	`CONTROL_PLANE_READ`	Xem chi tiết một flag, gồm group summary nếu flag đang được gán group.
PATCH	`/v1/projects/{projectKey}/flags/{flagKey}`	`FLAG_MANAGE`	Cập nhật metadata và config runtime: `name`, `description`, `status`, `servingMode`, `killSwitch`.
POST	`/v1/projects/{projectKey}/flags/{flagKey}/archive`	`FLAG_LIFECYCLE_MANAGE`	Archive flag; data plane sẽ trả tắt với reason `FLAG_ARCHIVED`.
POST	`/v1/projects/{projectKey}/flags/{flagKey}/restore`	`FLAG_LIFECYCLE_MANAGE`	Khôi phục flag đã archive về vòng đời active.
DELETE	`/v1/projects/{projectKey}/flags/{flagKey}`	`FLAG_LIFECYCLE_MANAGE`	Soft-delete flag, trả `204 No Content`.
POST	`/v1/projects/{projectKey}/flags/{flagKey}/restore-deleted`	`FLAG_LIFECYCLE_MANAGE`	Khôi phục flag đã soft-delete.

Rule API

Bảng 2.30: Đặc tả Rule API

Method	Endpoint	Quyền	Mục đích và dữ liệu chính
GET	`/v1/projects/{projectKey}/flags/{flagKey}/rules`	`CONTROL_PLANE_READ`	Liệt kê rule của flag. Query hỗ trợ `type` và pagination.
PUT	`/v1/projects/{projectKey}/flags/{flagKey}/rules`	`RULE_MANAGE`	Thay thế toàn bộ rule set bằng mảng `rules`. Mỗi rule gồm `type`, `priority`, `enabled`, `parameters`.

Flag Group API

Bảng 2.31: Đặc tả Flag Group API

Method	Endpoint	Quyền	Mục đích và dữ liệu chính
GET	`/v1/projects/{projectKey}/groups`	`CONTROL_PLANE_READ`	Liệt kê group. Query hỗ trợ `search`, `environmentKey` và pagination.
POST	`/v1/projects/{projectKey}/groups`	`GROUP_MANAGE`	Tạo group với `key` và `name`. `key` là bất biến trong project.
PATCH	`/v1/projects/{projectKey}/groups/{groupKey}`	`GROUP_MANAGE`	Đổi `name` của group.
DELETE	`/v1/projects/{projectKey}/groups/{groupKey}`	`GROUP_MANAGE`	Xóa group, trả `204 No Content`.
PUT	`/v1/projects/{projectKey}/groups/{groupKey}/config`	`GROUP_KILL_SWITCH`	Cập nhật `killSwitch` cho một `environmentKey`.
PUT	`/v1/projects/{projectKey}/flags/{flagKey}/group`	`GROUP_ASSIGN`	Gán flag vào group bằng body `groupKey`; trả lại flag response mới.
DELETE	`/v1/projects/{projectKey}/flags/{flagKey}/group`	`GROUP_ASSIGN`	Gỡ flag khỏi group; trả lại flag response mới.

Sample User API

Bảng 2.32: Đặc tả Sample User API

Method	Endpoint	Quyền	Mục đích và dữ liệu chính
GET	`/v1/projects/{projectKey}/sample-users`	`CONTROL_PLANE_READ`	Liệt kê sample user. Query hỗ trợ `search`, `role` và pagination.
POST	`/v1/projects/{projectKey}/sample-users`	`SAMPLE_USER_MANAGE`	Tạo sample user với `displayName`, `targetingKey`, `userId`, `roles`, `attributes`.
DELETE	`/v1/projects/{projectKey}/sample-users/{targetingKey}`	`SAMPLE_USER_MANAGE`	Xóa sample user theo `targetingKey`, trả `204 No Content`.

2.7.2 Data plane API

Data plane chỉ có một endpoint chính: `POST /v1/evaluate`. Endpoint này được đánh dấu public trong controller, không dùng bearer token quản trị, và được tối ưu cho tính xác định, fail-closed và an toàn khi thiếu cấu hình. Đây là API mà Demo Application gọi thông qua `@ffp/js-sdk` để quyết định có hiển thị một tính năng hay không.

Bảng 2.33: Đặc tả Evaluation API

Method	Endpoint	Quyền	Mục đích và dữ liệu chính
POST	`/v1/evaluate`	Public	Trả quyết định runtime cho một `projectKey`, `flagKey`, `environmentKey` tùy chọn và `context`. Response luôn có dạng evaluation result với `enabled`, `variant`, `reason`.

Request body của Evaluation API gồm khóa cấu hình và context runtime:

Đoạn mã 2.10: Ví dụ request Evaluation API

```
{
  "projectKey": "demo-project",
  "environmentKey": "production",
  "flagKey": "new-checkout",
  "context": {
    "targetingKey": "demo-user-beta",
    "userId": "demo-user-beta",
    "roles": ["beta-tester"],
    "attributes": {
      "plan": "pro",
      "country": "VN"
    }
  }
}
```

Response body trả lại quyết định đã được giải thích bằng reason code:

Đoạn mã 2.11: Ví dụ response Evaluation API

```
{
  "projectKey": "demo-project",
  "flagKey": "new-checkout",
  "enabled": true,
  "variant": "on",
  "reason": "ROLE_MATCH",
  "matchedRuleId": "rule_123"
}
```

Nếu project, environment, flag hoặc config theo environment không tồn tại, API không trả lỗi `404` kiểu control plane mà trả kết quả an toàn: `enabled = false`, `variant = "off"`, `reason = "NOT_FOUND"`. Nếu request hợp lệ nhưng thiếu context cần thiết cho percentage rollout, engine trả `INVALID_CONTEXT`. Nếu có lỗi bất ngờ trong đường evaluate, service bắt lỗi và trả

ERROR theo nguyên tắc fail-closed. Các reason code hiện có gồm GLOBAL_ON, FLAG_DISABLED, FLAG_ARCHIVED, GROUP_KILL_SWITCH, KILL_SWITCH, USER_ALLOWLIST, ROLE_MATCH, PERCENTAGE_ROLLOUT, DEFAULT_OFF, NOT_FOUND, INVALID_CONTEXT và ERROR.

Thứ tự ưu tiên của data plane là: archived flag, disabled config, group kill switch, flag kill switch, global on, ordered enabled rules và default off. Vì vậy một rule rollout không thể vô tình bật flag khi flag đã bị archive, disabled hoặc bị kill switch.

2.7.3 Monitoring API

Monitoring API gồm audit/history và evaluation statistics. Audit log là nguồn append-only để truy vết actor, action, target, request ID và snapshot trước/sau; flag history là projection từ audit log, không tạo nguồn sự thật thứ hai. Statistics chỉ lưu dữ liệu aggregate theo flag, environment, reason, enabled/disabled và bucket giờ UTC, không lưu raw evaluation context hay định danh người dùng cuối. Ngoài ra backend có health probe công khai GET/v1/health.

Audit Log API

Bảng 2.34: Đặc tả Audit Log và Flag History API

Method	Endpoint	Quyền	Mục đích và dữ liệu chính
GET	/v1/projects/{projectKey}/audit-logs	CONTROL_PLANE_READ	Liệt kê audit log của project. Query hỗ trợ targetType, targetKey, actor, action, from, to và pagination.
GET	/v1/projects/{projectKey}/flags/{flagKey}/history	CONTROL_PLANE_READ	Lịch sử cấu hình của một flag, được projection từ audit log liên quan đến flag, config và rule. Query hỗ trợ pagination và sort=createdAt.

Evaluation Statistics API

Bảng 2.35: Đặc tả Evaluation Statistics API

Method	Endpoint	Quyền	Mục đích và dữ liệu chính
GET	`/v1/projects/{projectKey}/stats/flags`	`CONTROL_PLANE_READ`	Thống kê tổng hợp theo các flag trong project. Query hỗ trợ `environmentKey`, `from`, `to`, pagination và sort theo `totalEvaluations`, `enabledCount`, `disabledCount`, `flagKey`.
GET	`/v1/projects/{projectKey}/flags/{flagKey}/stats`	`CONTROL_PLANE_READ`	Thống kê chi tiết một flag: tổng lượt evaluate, số enabled/disabled, phần trăm enabled, breakdown theo reason và bucket theo giờ. Query hỗ trợ `environmentKey`, `from`, `to`.

2.7.4 Validation và mô hình lỗi

Validation được thực hiện ở biên API bằng DTO, `class-validator`, `class-transformer` và global `ValidationPipe`. Pipe bật `whitelist`, `forbidNonWhitelisted` và `transform`, vì vậy field ngoài DTO bị từ chối, query string như `limit` được chuyển sang số, và lỗi validation được chuẩn hóa trước khi vào service. Các khóa nghiệp vụ như `projectKey`, `flagKey`, `groupKey` và `environmentKey` dùng regex:

Đoạn mã 2.12: Định dạng key nghiệp vụ

```
^[a-z0-9][a-z0-9-]{1,62}[a-z0-9]$
```

Quy tắc này tương ứng độ dài 3–64 ký tự, chỉ gồm chữ thường, chữ số và dấu gạch ngang, không bắt đầu hoặc kết thúc bằng dấu gạch ngang. Các DTO còn giới hạn độ dài `name` tối đa 120 ký tự, `description` tối đa 1000 ký tự, pagination `limit` từ 1 đến 100, `offset` không âm, thời gian `from`/`to` theo ISO-8601 và enum phải thuộc giá trị hợp lệ.

Control-plane error response có dạng thống nhất:

Đoạn mã 2.13: Mẫu error response của control plane

```
{
  "code": "VALIDATION_ERROR",
  "message": "Request validation failed.",
  "details": [
    {
```

```
      "field": "projectKey",
      "message": "projectKey must be a valid 3-64 character key."
    }
  ],
  "requestId": "req_123"
}
```

Các mã lỗi chuẩn gồm `VALIDATION_ERROR`, `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`, `CONFLICT` và `INTERNAL_ERROR`. `UNAUTHORIZED` dùng khi thiếu hoặc sai bearer token demo; `FORBIDDEN` dùng khi role đã xác thực nhưng không có permission; `CONFLICT` dùng cho xung đột như khóa đã tồn tại; `INTERNAL_ERROR` che giấu chi tiết lỗi nội bộ nhưng vẫn log kèm request ID. Riêng Evaluation API có thêm mô hình lỗi domain bằng reason code trong response 200 để giữ fail-closed, ví dụ `NOT_FOUND`, `INVALID_CONTEXT` hoặc `ERROR` đều trả `enabled = false` và `variant = off` thay vì để client tự suy diễn.

2.8 Hiệu năng, nhất quán và bảo mật

Mục này tổng hợp các quyết định phi chức năng quan trọng của hệ thống dựa trên hiện trạng triển khai trong source code. Trọng tâm là cách data plane giữ độ trễ thấp nhưng vẫn fail-closed, cách control plane bảo vệ các thao tác thay đổi cấu hình, và cách hệ thống duy trì khả năng truy vết khi cấu hình feature flag thay đổi. Các chi tiết giao diện và kết quả chạy thử được trình bày ở chương kết quả.

2.8.1 Hiệu năng data plane: snapshot cache, TTL và thống kê

Evaluation API là đường chạy thường xuyên nhất vì demo application và JavaScript SDK gọi endpoint này để quyết định bật/tắt tính năng. Để giảm số lần đọc PostgreSQL lặp lại, backend cache *evaluation snapshot* thay vì cache quyết định cuối cùng theo từng người dùng. Snapshot chứa phần cấu hình cần thiết để engine đánh giá một flag trong một environment: trạng thái lifecycle của flag, trạng thái cấu hình theo environment, serving mode, kill switch, group kill switch nếu có, và danh sách rule đã nạp từ database. Khi cache miss, `EvaluationRepository.findSnapshot` đọc lại cấu hình từ PostgreSQL; khi cache hit, engine vẫn tự tính kết quả dựa trên evaluation context của request.

Cách cache snapshot này phù hợp với yêu cầu an toàn của mini project. Hệ thống không ưu tiên cache final decision theo user context, vì quyết định cuối cùng phụ thuộc vào `targetingKey`, `userId`, `roles` và các thuộc tính đi kèm. Nếu cache trực tiếp kết quả cuối, hệ thống dễ phải đưa định danh người dùng vào cache key hoặc lưu dữ liệu context nhạy cảm. Do đó thiết kế hiện tại đánh đổi một phần hiệu năng để giữ mô hình đơn giản hơn, ít rủi ro dữ liệu cá nhân hơn, và không làm thay đổi tính xác định của engine.

Cache provider được đóng gói sau interface `EvaluationSnapshotCache`. Source code hiện hỗ trợ ba lựa chọn: `memory`, `redis` và `none`. Mặc định hệ thống dùng in-memory cache với TTL 30 giây thông qua `DEFAULT_EVALUATION_CACHE_TTL_MS`; giá trị này có thể cấu hình bằng biến môi trường `EVALUATION_CACHE_TTL_MS`. In-memory cache phù hợp cho demo, kiểm thử cục bộ và một backend instance. Redis được triển khai như provider mở rộng cho trường hợp nhiều backend instance cần chia sẻ snapshot hoặc cần invalidation phân tán; nếu không muốn cache, provider `none` cho phép tắt cache mà không đổi API đánh giá.

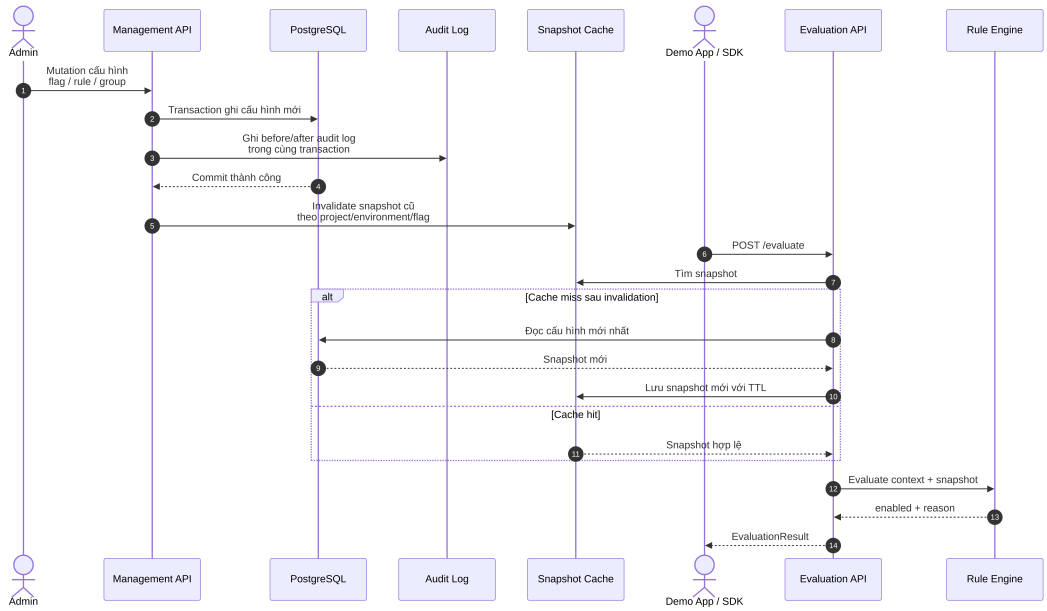
Ngoài cache, backend còn ghi thống kê evaluation theo hướng không chặn response. Sau khi engine trả kết quả, `EvaluationMetricsService` ghi tăng bộ đếm theo project, environment, flag, reason, enabled result và bucket thời gian theo giờ UTC. Nếu ghi thống kê thất bại, hệ thống chỉ log cảnh báo và vẫn giữ nguyên response evaluation. Điều này giúp statistics dashboard có dữ liệu quan sát vận hành nhưng không biến thống kê thành điểm lỗi của data plane.

2.8.2 Cache key, invalidation và tính nhất quán cấu hình

Cache key được thiết kế theo dữ liệu cấu hình, không theo dữ liệu người dùng. Hàm tạo cache key ghép bốn thành phần: prefix `evaluation-snapshot`, `projectKey`, `environmentKey` và `flagKey`. Khi request không truyền environment, backend dùng scope mặc định `__default__` để phân biệt với các environment cụ thể. Nhờ đó snapshot của staging, production hoặc default environment không bị dùng nhầm lẫn cho nhau. Evaluation context không nằm trong cache key vì hệ thống chỉ cache snapshot cấu hình, không cache quyết định cuối cùng.

Invalidation được kích hoạt khi một mutation có thể làm thay đổi kết quả evaluation. Các thay đổi như cập nhật lifecycle của flag, bật/tắt config theo environment, đổi serving mode, đổi flag kill switch, thay rule, đổi priority rule, assign/unassign flag group, hoặc cập nhật group kill switch đều cần loại bỏ snapshot liên quan. `EvaluationCacheInvalidator` hỗ trợ invalidation một flag hoặc nhiều flag theo project, flag key và optional environment key. Nguyên tắc triển

khai là thà invalidation hơi rộng còn hơn giữ snapshot lỗi thời; ví dụ khi không chứng minh được scope environment, cache của default alias cũng được xóa để tránh stale read.



Hình 2.7: Luồng cache invalidation và bảo đảm tính nhất quán cấu hình

TTL đóng vai trò lớp bảo vệ phụ trong trường hợp một lần invalidation bị bỏ sót hoặc provider cache gặp lỗi. TTL không thay thế invalidation sau mutation cấu hình, vì yêu cầu của hệ thống là thay đổi cấu hình phải được phản ánh nhanh trên data plane. Trong phạm vi một backend instance, mutation ghi database xong rồi invalidation cache giúp hệ thống gần với strong consistency sau thao tác quản trị. Khi mở rộng nhiều instance, Redis hoặc cơ chế pub/sub sẽ cần thiết hơn để giảm độ trễ đồng bộ giữa các instance.

Tính nhất quán của cấu hình cũng được bảo vệ bằng transaction và audit log. Các mutation quan trọng ghi thay đổi cấu hình và audit entry trong cùng một transaction, tránh tình trạng cấu hình đã đổi nhưng không có bằng chứng truy vết. Evaluation path fail-closed: nếu không tìm thấy project, environment, flag hoặc config thì trả `enabled=false` với `reason=NOT_FOUND`; nếu quá trình đọc snapshot hoặc đánh giá gặp lỗi bất thường thì trả `enabled=false` với `reason=ERROR`. Cache chỉ là tối ưu hiệu năng, không được phép thay đổi thứ tự ưu tiên hoặc kết quả xác định của engine.

2.8.3 Trust boundary của evaluation context và SDK

Evaluation API được đánh dấu public vì ứng dụng tích hợp cần gọi endpoint này ở runtime. Do đó evaluation context do client gửi lên phải luôn được xem là dữ liệu không tin cậy. Engine

có thể dùng context để quyết định rollout, nhưng không dùng context thay cho xác thực hoặc phân quyền nghiệp vụ. Trong demo, các trường như `userId`, `roles` hoặc attribute mô phỏng người dùng giúp minh họa user allowlist, role targeting và percentage rollout; trong hệ thống production, các trường nhạy cảm nên được xác nhận bởi backend nghiệp vụ hoặc hệ thống định danh đáng tin cậy.

Thiết kế rollout ưu tiên định danh ổn định nhưng không nhạy cảm. Rule percentage rollout dùng `targetingKey` cùng `projectKey` và `flagKey` để tính bucket bằng stable hashing. Cùng một user context hợp lệ sẽ nhận cùng một kết quả cho cùng một flag, trong khi hệ thống không cần lưu trạng thái rollout riêng cho từng người dùng. Nếu thiếu `targetingKey` khi có percentage rollout rule, engine trả kết quả fail-closed với `reason=INVALID_CONTEXT`.

Ranh giới giữa client-side và server-side evaluation cũng cần được hiểu rõ. Client-side evaluation thuận tiện cho việc ẩn/hiện UI, còn server-side evaluation an toàn hơn cho chức năng nhạy cảm. Mini project dùng API tập trung ở backend để giữ logic đánh giá trong một nơi và để demo app cũng như SDK không tự triển khai lại rule engine. Tuy nhiên, feature flag chỉ quyết định phát hành tính năng; nó không thay thế authorization. Các endpoint quan trọng vẫn phải kiểm tra quyền ở backend ngay cả khi UI bị ẩn bởi feature flag.

Để minh họa ranh giới này, Demo Application bổ sung demo backend API dưới `/api/demo/features/*`. Frontend vẫn dùng SDK để ẩn/hiện UI, trong khi các thao tác demo có thể gọi backend endpoint tương ứng với feature flag trước khi trả dữ liệu hoặc thực thi hành vi của tính năng. Demo backend tiếp tục dùng `@ffp/js-sdk` để hỏi Evaluation API trước khi trả dữ liệu tính năng. Nếu flag trả `enabled=false`, hoặc SDK gặp timeout/lỗi mạng/response không hợp lệ, endpoint fail-closed với `403 FEATURE_DISABLED`. Điều này chứng minh frontend hiding không phải lớp bảo vệ duy nhất và cùng một contract evaluation có thể dùng ở cả browser lẫn server-side integration.

JavaScript SDK hỗ trợ tích hợp an toàn hơn bằng cách đóng gói các hàm `evaluate`, `isEnabled` và `getVariant`. SDK validate response từ backend, giới hạn thời gian chờ mặc định 1500 ms, và khi gặp lỗi request, timeout, network, HTTP, JSON hoặc schema response thì trả kết quả fail-closed: `enabled=false`, `variant=off`, `reason=ERROR`. Nhờ đó ứng dụng tích hợp không phải tự xử lý quá nhiều trường hợp lỗi và vẫn giữ mặc định an toàn.

2.8.4 Bảo vệ Management API bằng RBAC và validation

Management API thuộc control plane vì các endpoint này thay đổi project, feature flag, rule, group, sample user, kill switch và các cấu hình liên quan. Khác với Evaluation API public, các controller quản trị dùng bearer token demo và decorator `RequirePermissions` để yêu cầu quyền phù hợp. Backend là nơi quyết định quyền cuối cùng; dashboard chỉ ẩn hoặc vô hiệu hóa thao tác không được phép nhằm cải thiện trải nghiệm người dùng, không phải lớp bảo vệ chính.

RBAC demo phân biệt ba vai trò. `admin` có toàn quyền trong phạm vi demo, bao gồm quản lý project, flag, lifecycle, rule, group, group kill switch, sample user, audit log và statistics. `developer` có quyền đọc control plane và thực hiện các thao tác cấu hình thường gặp như quản lý flag, rule và assign group, nhưng không mặc định có quyền cho các thao tác nhạy cảm hơn như quản lý project hoặc bật group kill switch. `viewer` chỉ có quyền đọc. Actor được backend resolve từ bearer token thông qua `DemoIdentityService`; actor ghi vào audit log không lấy trực tiếp từ body request, nhờ đó giảm nguy cơ giả mạo lịch sử thao tác.

Validation DTO và error handling là lớp bảo vệ bổ sung cho control plane. Các khóa project, flag, environment và group được validate ở boundary API; các request phân trang, time range, rule parameters và mutation body được chuẩn hóa trước khi đi vào service. Khi quyền không đủ hoặc dữ liệu không hợp lệ, backend trả lỗi có cấu trúc thay vì để lỗi nội bộ rò rỉ ra client. Cách này giúp API dễ demo, dễ kiểm thử và phù hợp với yêu cầu clean APIs, validation và error handling của đề bài.

2.8.5 Auditability và khả năng truy vết thay đổi

Auditability là yêu cầu xuyên suốt của hệ thống. Mỗi mutation quan trọng cần ghi actor, action, target type, target key, request ID, timestamp, metadata và snapshot trước/sau nếu có. Bảng `AuditLogEntry` được thiết kế append-only ở mức ứng dụng: service chỉ tạo bản ghi mới cho sự kiện thay đổi, không dùng audit log như bảng trạng thái hiện tại. Dữ liệu này hỗ trợ demo rollback reasoning, kiểm thử, phân tích sự cố và mở rộng sang quy trình phê duyệt trong tương lai [10].

Hệ thống cũng dùng audit log như nguồn cho configuration history. Endpoint history của flag truy vấn các audit entry liên quan đến feature flag, flag environment config và flag rule để hiển thị lịch sử thay đổi cấu hình. Cách làm này tránh tạo một hệ thống versioning song song trong phạm vi mini project, đồng thời vẫn đáp ứng nhu cầu xem lại ai đã thay đổi gì, khi nào,

trước/sau ra sao và request nào tạo ra thay đổi đó.

Audit log cần được bảo vệ như dữ liệu quản trị vì snapshot cấu hình có thể tiết lộ rule targeting, rollout strategy hoặc thông tin vận hành. Vì vậy quyền đọc audit/history nằm trong control-plane read permission và đi qua RBAC demo. Khi kết hợp audit log với statistics dashboard, người vận hành có thể trả lời hai câu hỏi khác nhau: cấu hình đã thay đổi như thế nào, và sau thay đổi đó runtime evaluation đang trả những reason nào. Đây là nền tảng để trình diễn system thinking của dự án: hiệu năng, tính nhất quán, bảo mật và khả năng quan sát được thiết kế cùng nhau thay vì xử lý rời rạc.

Chương 3

Kết quả thực hiện và đánh giá

3.1 Môi trường và cấu trúc triển khai

3.1.1 Cấu trúc mã nguồn

Mã nguồn được tổ chức theo mô hình npm workspace để tách rõ các thành phần nhưng vẫn quản lý chung trong một repository. Cấu trúc chính gồm `apps/backend` cho Backend API NestJS, `apps/admin` cho Admin Dashboard React, `apps/demo` cho Demo Application React và `packages/js-sdk` cho SDK JavaScript dùng bởi ứng dụng demo. Cách tổ chức này giúp các thành phần dùng chung quy trình cài đặt, build, kiểm thử và triển khai, đồng thời giảm rủi ro lệch API contract giữa control plane và data plane [4, 11].

Ở Backend API, mã nguồn được chia theo module nghiệp vụ như project, feature flag, flag group, rule, evaluation, statistics, audit log, sample user và auth. Các controller chỉ tiếp nhận request và trả response; service/repository xử lý nghiệp vụ và truy cập PostgreSQL thông qua Prisma. Các phần dùng chung như validation, request context, exception filter và permission guard được đặt trong các module `common` và `auth` để tránh lặp lại xử lý ở từng endpoint.

Admin Dashboard được xây dựng theo hướng component-based với các trang project, danh sách flag, cấu hình rule, flag group, audit log và statistics. Những thành phần dùng lặp lại như badge trạng thái, dialog xác nhận, data state, history panel và API client được tách riêng để giao diện nhất quán và dễ mở rộng. Demo Application là một ứng dụng React mô phỏng storefront thương mại điện tử, đồng thời có thư mục `apps/demo/server` cho demo backend API. Frontend và demo backend cùng tích hợp SDK trong `packages/js-sdk` để đánh giá flag,

kiểm tra response và áp dụng mặc định fail-closed khi API lỗi [12].

3.1.2 Môi trường phát triển

Toàn bộ hệ thống sử dụng TypeScript trên Node.js. Backend dùng NestJS, frontend dùng React/Vite, còn npm workspaces quản lý các package trong `apps` và `packages`. Các lệnh phát triển ở cấp root gồm `npm run dev:backend`, `npm run dev:admin`, `npm run dev:demo`, `npm run build`, `npm run test` và `npm run lint`. Việc thống nhất công cụ giúp quá trình phát triển, kiểm thử và đóng gói các thành phần diễn ra đồng nhất.

Cơ sở dữ liệu chính là PostgreSQL, được truy cập thông qua Prisma ORM. Prisma Schema định nghĩa các bảng, khóa ngoại, chỉ mục, quan hệ và kiểu enum; PostgreSQL JSONB được dùng cho tham số rule, thuộc tính user context và snapshot before/after của audit log [13]. Backend bật `ValidationPipe`, Swagger/OpenAPI, CORS theo biến môi trường và exception filter để chuẩn hóa lỗi. Jest được dùng cho unit test và e2e/integration test, tập trung vào rule evaluation, cache, statistics, RBAC, persistence và các luồng demo quan trọng.

3.1.3 Database migration và seed data

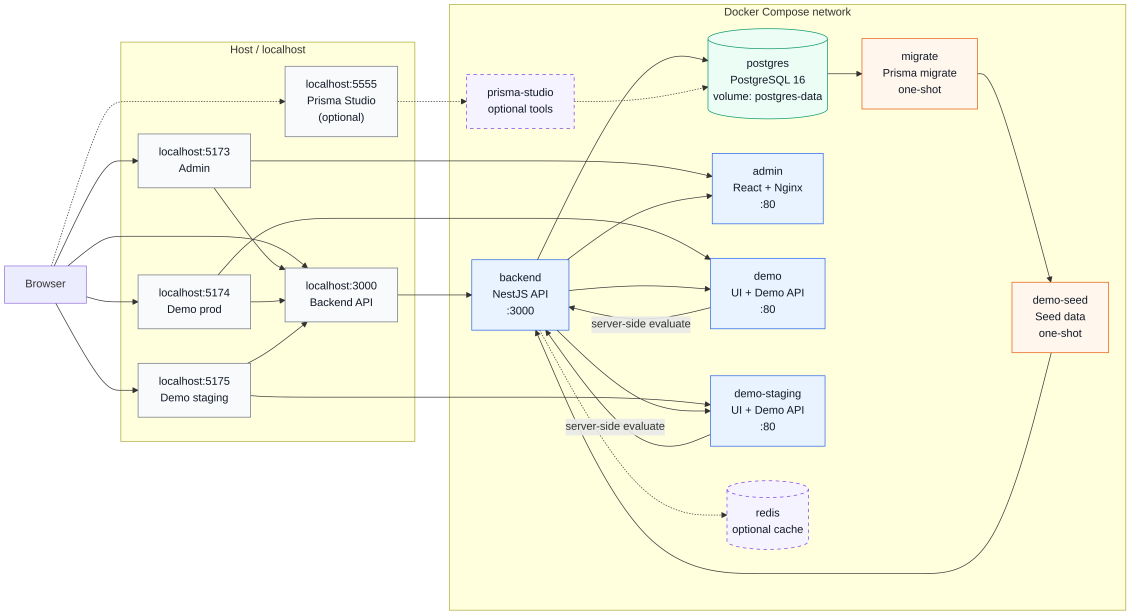
Schema hệ thống nằm tại `apps/backend/prisma/schema.prisma` và được triển khai bằng Prisma Migrate. Các migration hiện có tạo và mở rộng các nhóm bảng cho project, environment, feature flag, cấu hình theo môi trường, rule, flag group, sample user, audit log và evaluation metrics. Những ràng buộc như unique key theo project, chỉ mục truy vấn, cascade/restrict relation và soft delete được khai báo trong schema để bảo vệ tính nhất quán dữ liệu.

Seed script `apps/backend/prisma/seed.ts` được thiết kế không phá hủy dữ liệu đang có: script chỉ tạo bản ghi còn thiếu và không reset các chỉnh sửa demo hiện tại. Dữ liệu mẫu bao gồm project `demo-project`, các environment `production`, `staging`, `development`, các nhóm flag dùng cho kịch bản demo, nhiều feature flag mẫu và các user mẫu đại diện cho các vai trò khác nhau. Các rule user allowlist, role targeting và percentage rollout được seed kèm audit log ban đầu, nhờ đó hệ thống có dữ liệu sẵn để kiểm thử và trình diễn ngay sau khi khởi tạo.

3.1.4 Triển khai bằng Docker Compose

Hệ thống có thể chạy bằng Docker Compose để tái tạo đầy đủ môi trường trình diễn. Tệp `docker-compose.yml` định nghĩa PostgreSQL, backend, admin, demo, `demo-staging`, các job

`migrate` và `demo-seed`; ngoài ra có profile tùy chọn cho Prisma Studio và Redis. Hình 3.1 minh họa các service chính và quan hệ phụ thuộc khi triển khai bằng Docker Compose.



Hình 3.1: Các service chính trong quy trình triển khai bằng Docker Compose

Các container giao tiếp qua mạng nội bộ do Docker Compose tạo. Backend kết nối PostgreSQL bằng tên service `postgres`; Admin Dashboard và phần browser của Demo Application gọi Backend API thông qua biến `VITE_API_BASE_URL`. Riêng demo backend chạy trong container `demo` gọi platform backend bằng `DEMO_SERVER_API_BASE_URL=http://backend:3000/v1`, vì `localhost` bên trong container demo không trỏ tới service backend. Service `postgres` dùng volume `postgres-data` để giữ dữ liệu sau khi container khởi động lại. Những cấu hình như database URL, cổng API, origin frontend, cache provider/TTL và token demo RBAC được đặt ngoài mã nguồn để cùng một image có thể chạy trên nhiều môi trường [9, 11].

3.1.5 Quy trình khởi chạy hệ thống

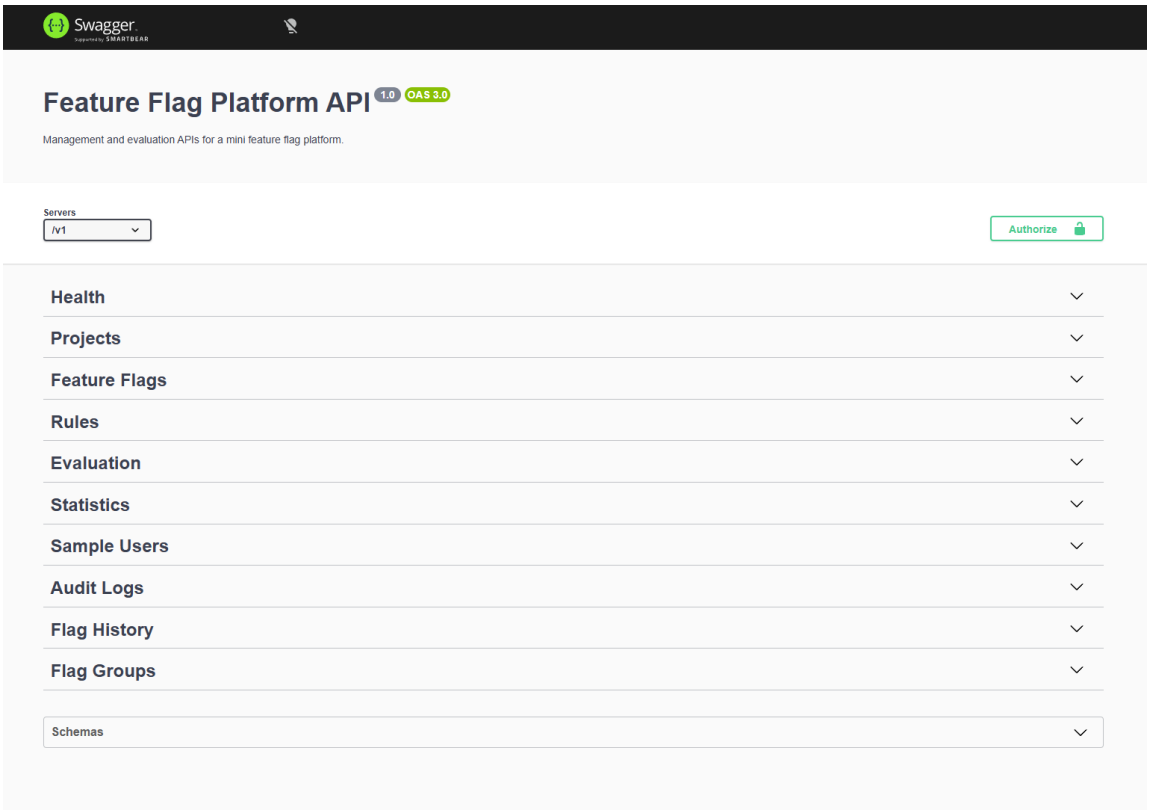
Quy trình khởi chạy được chuẩn hóa theo thứ tự phụ thuộc. Trên môi trường phát triển cục bộ, lập trình viên cài package bằng `npm install`, chuẩn bị biến môi trường, chạy PostgreSQL, sau đó dùng các script workspace để chạy backend, admin và demo. Với Docker Compose, PostgreSQL được kiểm tra healthcheck trước; job `migrate` chạy `prisma migrate deploy`; job `demo-seed` sinh dữ liệu mẫu; cuối cùng các service backend, admin, demo và demo staging được khởi động khi các bước trước hoàn tất. Service demo chạy Node server thay vì chỉ phục vụ static file, nhờ đó vừa serve frontend đã build vừa xử lý `/api/demo/*`.

Sau khi hệ thống chạy, trạng thái được xác nhận qua endpoint `/v1/health`, Swagger UI của Backend API, Admin Dashboard, Demo Application và `/api/demo/health`. Các kiểm tra thực tế gồm mở danh sách project/flag, xem audit log, gọi evaluation cho các user mẫu, gọi một endpoint theo mẫu `/api/demo/features/{featureKey}`, và bật/tắt một số cấu hình để xác nhận Dashboard, Evaluation API, SDK, demo backend và dữ liệu PostgreSQL hoạt động đồng bộ. Quy trình này giúp môi trường luôn sẵn sàng cho kiểm thử, trình diễn và đánh giá mà không cần cấu hình thủ công nhiều lần.

3.2 Kết quả xây dựng

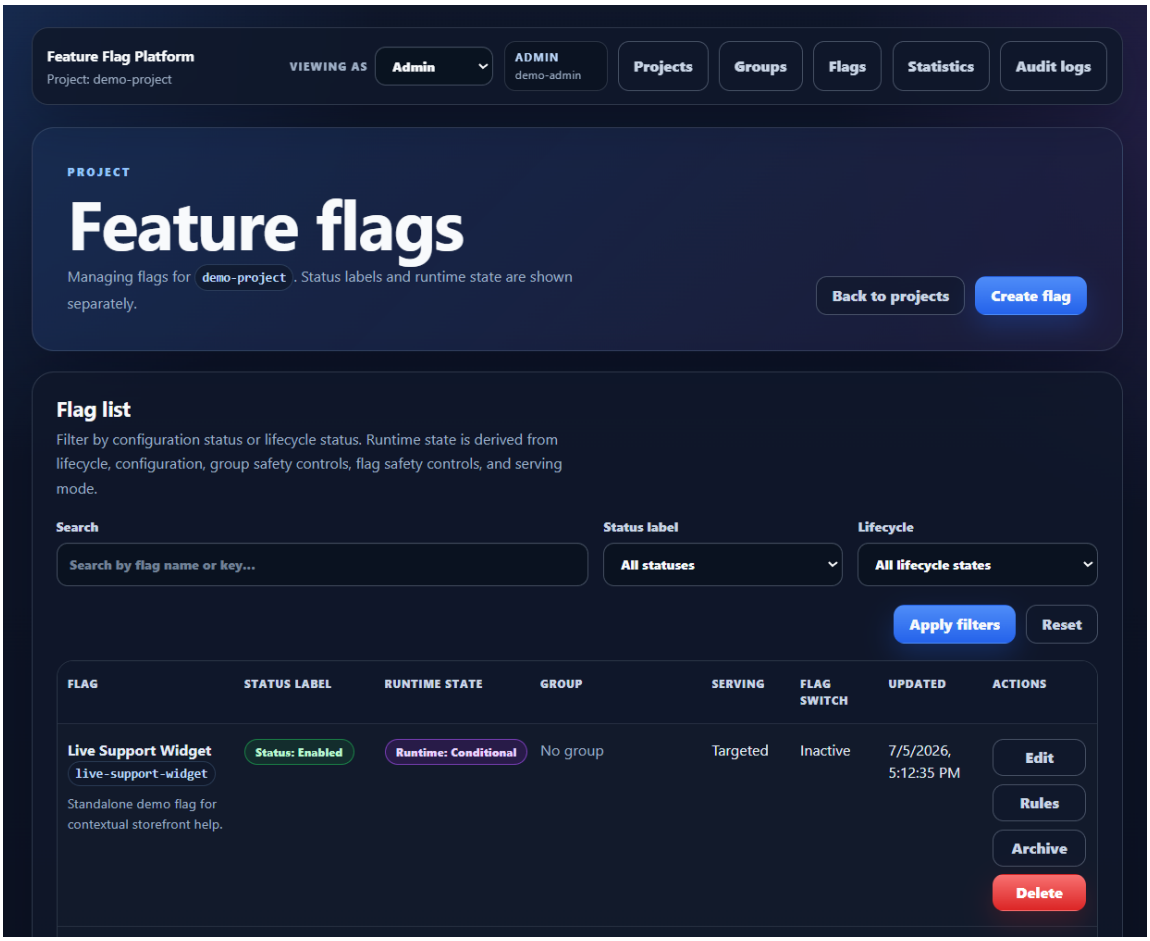
Kết quả xây dựng của hệ thống được lưu tại GitHub repository: `https://github.com/fabyanbui/feature-flag-platform`. Các minh chứng chi tiết về Backend API, Frontend Dashboard, Demo Application và SDK sẽ được trình bày chi tiết trong phần demo.

3.2.1 Backend API



Hình 3.2: Minh chứng kết quả xây dựng Backend API

3.2.2 Frontend Dashboard

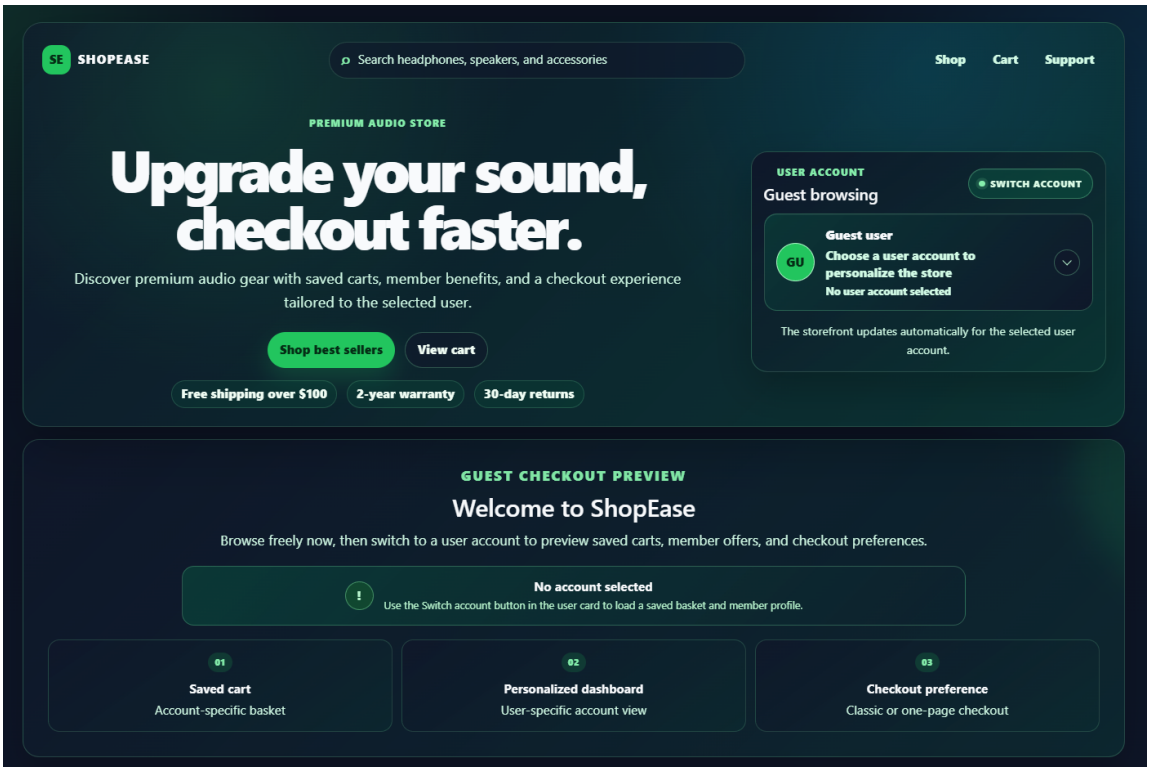


Hình 3.3: Minh chứng kết quả xây dựng Frontend Dashboard

3.2.3 Demo Application

Demo Application đã được mở rộng từ ứng dụng frontend thuần thành storefront có kèm demo backend API nhẹ. Mỗi feature flag trong demo có một endpoint GET tương ứng dưới `/api/demo/features/*`; endpoint này dùng `@ffp/js-sdk` ở phía server để gọi Evaluation API trước khi trả dữ liệu tính năng. Nếu flag tắt, bị archive, bị group/flag kill switch hoặc SDK không xác minh được trạng thái bật, demo backend trả `403 FEATURE_DISABLED`.

Một số thao tác có sẵn trong giao diện storefront được nối với endpoint backend tương ứng với feature flag, không cần thêm panel minh họa riêng. Nhờ đó phần demo thể hiện đồng thời hai lớp tích hợp: frontend ẩn/hiện UI theo flag và backend API vẫn tự guard khi bị gọi trực tiếp.



Hình 3.4: Minh chứng kết quả xây dựng Demo Application

3.3 Kiểm thử và đánh giá

Kế hoạch kiểm thử của dự án tập trung vào ba mức: unit test để kiểm tra từng đơn vị nghiệp vụ quan trọng, E2E test để xác nhận các luồng API chạy qua NestJS application và PostgreSQL, và CI để tự động kiểm tra chất lượng khi có thay đổi mã nguồn. Các nhóm kiểm thử ưu tiên những rủi ro chính của hệ thống Feature Flag: evaluation phải xác định, mặc định an toàn, mutation phải có audit log, cache không làm sai kết quả, phân quyền phải được xử lý ở backend và SDK phải fail-closed khi lỗi.

3.3.1 Unit test

Unit test được triển khai bằng Jest trong hai workspace chính: `@ffp/backend` và `@ffp/js-sdk`. Ở backend, unit test kiểm tra Rule Evaluation Engine, stable rollout hash, evaluation service/-controller/repository, cache, invalidation, audit log, history, RBAC, DTO validation, exception filter, request context, repository và statistics service. Các test dùng mock hoặc dữ liệu tối thiểu để cô lập đơn vị cần kiểm tra, sau đó đối chiếu output với contract mong đợi như `enabled`, `reason`, error code, audit snapshot hoặc permission result.

Với JavaScript SDK, unit test kiểm tra client gọi Evaluation API, validate response, trả helper `isEnabled`/`getVariant`, xử lý timeout, HTTP error, response sai định dạng và fallback fail-closed. Nhóm test này bảo đảm ứng dụng khách không tự bật tính năng khi Evaluation API không khả dụng.

Bảng 3.1: Thống kê unit test đã triển khai

Workspace	Phạm vi kiểm thử chính	Test suite	Test case
`@ffp/backend`	Evaluation engine, rollout hash, cache, audit, RBAC, DTO, controller, service, repository, statistics	52	416
`@ffp/js-sdk`	SDK client, response validation, helper API, timeout, lỗi mạng và fallback fail-closed	1	21
Tổng	Unit test toàn workspace qua `npm run test - -runInBand`	53	437

3.3.2 Integration test và E2E test

Repository đã cấu hình Jest cho integration test qua `apps/backend/test/jest-integration.json` và script `npm run test:integration -workspace=@ffp/backend`. Ở trạng thái hiện tại, không có file `*.integration-spec.ts` riêng; các kiểm thử tích hợp được thể hiện chủ yếu qua nhóm E2E test trong `apps/backend/test`.

E2E test dùng `@nestjs/testing` để khởi tạo `AppModule`, áp dụng global validation pipe, exception filter, request context middleware và demo RBAC giống runtime. Các test gọi HTTP bằng `supertest`, thao tác dữ liệu qua Prisma và dọn dữ liệu bằng helper test database. Hiện có 10 file `*.e2e-spec.ts`, bao phủ health check, management API, vertical slice tạo project/flag/rule/evaluate, API hardening, demo flow, flag history, group kill switch, evaluation cache, evaluation statistics và server-resolved RBAC.

3.3.3 Luồng CI trong CI/CD

Luồng CI được triển khai bằng GitHub Actions tại `.github/workflows/ci.yml`. Workflow chạy khi push hoặc pull request vào nhánh `develop` và `main`, đồng thời hỗ trợ chạy thủ công bằng `workflow_dispatch`. Job `validate` sử dụng Ubuntu 24.04, Node.js 22.22.2 và PostgreSQL 16 service để mô phỏng môi trường kiểm thử có cơ sở dữ liệu thật.

Các bước CI lần lượt gồm kiểm tra whitespace bằng `git diff -check`, cài dependencies bằng `npm ci`, validate và generate Prisma client, chạy lint, chạy unit test backend, build toàn bộ

workspace, apply database migration, chạy seed hai lần để kiểm tra idempotency, chạy integration test và cuối cùng chạy backend E2E test. Luồng này giúp phát hiện sớm lỗi định dạng, lỗi schema, lỗi TypeScript/build, lỗi unit test và lỗi end-to-end trước khi thay đổi được hợp nhất.

3.4 Các kịch bản demo

Các kịch bản dưới đây sẽ được trình bày cụ thể trong phần demo. Mục tiêu là cho thấy hệ thống có thể thay đổi hành vi ứng dụng bằng cấu hình Feature Flag, đồng thời vẫn giữ được tính xác định, khả năng rollback nhanh, audit và quan sát vận hành.

3.4.1 Quan sát project, flag và ứng dụng demo

Kịch bản mở đầu giới thiệu project `demo-project`, các feature flag và nhóm flag đã seed, sau đó mở Demo Application để cho thấy ứng dụng đang nhận quyết định từ Evaluation API thông qua SDK.

3.4.2 Bật và tắt tính năng toàn cục

Kịch bản này thay đổi cấu hình serving mode hoặc trạng thái runtime của một flag đang bật toàn cục. Demo Application được evaluate lại với cùng user context để quan sát tính năng xuất hiện hoặc bị tắt mà không cần build/deploy lại mã nguồn.

3.4.3 Targeting theo role

Kịch bản này chuyển giữa các user mẫu như beta customer, regular customer và shop admin. Demo sẽ cho thấy cùng một flag có thể bật cho nhóm người dùng có role phù hợp và tắt cho nhóm còn lại, đồng thời hiển thị `projectKey`, `flagKey`, `enabled` và `reason`.

3.4.4 Rollout theo phần trăm

Kịch bản này dùng nhiều tài khoản mẫu có `targetingKey` ổn định để minh họa percentage rollout. Một số tài khoản nhận tính năng mới, số khác vẫn dùng trải nghiệm cũ; khi evaluate lại cùng context, kết quả không thay đổi do stable hashing.

3.4.5 Kill switch cấp flag và group

Kịch bản này bật kill switch ở cấp một flag hoặc cấp group tương ứng. Demo sẽ nhấn mạnh rằng trạng thái runtime chuyển sang Off ngay lập tức, trong khi lifecycle/configuration của từng flag vẫn được giữ lại để có thể khôi phục nhanh.

3.4.6 Backend API guard cho tính năng

Kịch bản này chứng minh Feature Flag không chỉ ẩn giao diện ở frontend. Demo Application có các endpoint `GET/api/demo/features/{featureKey}` tương ứng với các feature flag đã seed. Khi người dùng thực hiện một thao tác thuộc tính năng đang được flag bảo vệ, storefront gọi endpoint backend tương ứng. Demo backend dùng SDK để evaluate flag trước khi trả dữ liệu hoặc thực thi hành vi demo của tính năng.

Nếu flag đang bật, endpoint trả `200 OK` và UI hiển thị thông báo thao tác thành công. Sau đó, người trình bày có thể tắt flag tương ứng trên Admin Dashboard hoặc bật group kill switch của nhóm chứa flag rồi gọi lại endpoint bằng `curl`. Kết quả mong đợi là `403 FEATURE_DISABLED` kèm evaluation result, cho thấy người dùng không thể bypass feature flag bằng cách gọi API trực tiếp.

3.4.7 Audit log, history và statistics

Kịch bản này thực hiện một số mutation cấu hình và nhiều lần evaluation từ Demo Application, sau đó mở Dashboard để xem audit log, configuration history và statistics. Phần demo sẽ cho thấy actor, action, request ID, before/after snapshot và số liệu aggregate theo flag/reason.

3.4.8 Fallback của JavaScript SDK

Kịch bản này mô phỏng Evaluation API lỗi hoặc timeout. SDK `@ffp/js-sdk` sẽ trả kết quả fail-closed với `enabled=false`, giúp Demo Application tiếp tục hoạt động ở trạng thái an toàn và phân biệt được lỗi client-local với quyết định từ backend.

3.4.9 Phân quyền Admin, Developer và Viewer

Kịch bản cuối cùng chuyển giữa ba vai trò demo trên Admin Dashboard. Viewer chỉ xem dữ liệu, Developer được thao tác trong phạm vi cấu hình được phép, còn Admin có quyền quản trị đầy đủ; các mutation bị từ chối sẽ được backend bảo vệ thay vì chỉ ẩn ở giao diện.

Chương 4

Kết luận

4.1 Tổng kết

4.1.1 Kết quả và đóng góp

Đề tài đã hoàn thành mục tiêu nghiên cứu Feature Flag và xây dựng một nền tảng quản lý phát hành tính năng an toàn ở phạm vi mini project. Các kết quả và đóng góp chính gồm:

- Hệ thống hóa kiến thức về Feature Flag, bao gồm vai trò trong release management, sự khác nhau giữa deployment và release, các loại flag phổ biến, rollout strategy, kill switch, audit log, caching, consistency và bảo mật endpoint evaluation.
- Xây dựng Backend API bằng NestJS, Prisma và PostgreSQL, hỗ trợ quản lý project, feature flag, rule, group, sample user, audit log, history, statistics và Evaluation API.
- Hiện thực Rule Evaluation Engine có thứ tự ưu tiên rõ ràng, deterministic percentage rollout bằng stable hashing, reason code ổn định và mặc định fail-closed khi không thể đánh giá an toàn.
- Xây dựng Admin Dashboard để quản lý project, flag, rule, group kill switch, audit log, configuration history, statistics và trạng thái quyền theo vai trò admin/developer/viewer.
- Xây dựng Demo Application và JavaScript SDK để chứng minh ứng dụng khách có thể gọi Evaluation API, hiển thị kết quả `enabled`/`reason` và bật/tắt tính năng mà không cần deploy lại.
- Mở rộng Demo Application với server-side feature gating, chứng minh cùng một quyết

định evaluation có thể được áp dụng ở cả frontend UI và backend API.

- Bổ sung các yêu cầu khuyến khích như evaluation snapshot cache, SDK, unit test, audit-backed history, RBAC demo, statistics dashboard, group kill switch và Docker Compose.
- Chuẩn bị seed data, kịch bản demo, CI workflow và bộ kiểm thử tự động để sản phẩm có thể chạy, kiểm chứng và trình bày được.

4.1.2 Bài học kinh nghiệm

Trong quá trình thực hiện đề tài, một số bài học kinh nghiệm quan trọng được rút ra như sau:

- Cần xác định phạm vi MVP rõ ràng trước khi mở rộng, vì Feature Flag là miền bài toán dễ phát triển sang A/B testing, realtime sync, approval workflow hoặc hệ thống phân tán.
- Thiết kế phải bắt đầu từ contract và precedence của Evaluation Engine; nếu thứ tự ưu tiên không rõ, kết quả bật/tắt sẽ khó giải thích và khó kiểm thử.
- Tách control plane và data plane giúp hệ thống dễ trình bày, dễ kiểm thử và hạn chế ảnh hưởng của các thao tác quản trị đến luồng evaluation runtime.
- Audit log nên được thiết kế ngay từ đầu trong cùng transaction với mutation để tránh mất dấu vết cấu hình và tạo nền tảng cho history.
- Cache chỉ an toàn khi có invalidation rõ ràng; với Feature Flag, snapshot cũ có thể trực tiếp làm sai quyết định release.
- Seed data và demo scenario cần được chuẩn bị cùng với tính năng để người xem thấy được giá trị thực tế thay vì chỉ nhìn thấy các API rời rạc.
- CI, unit test và E2E test giúp bảo vệ các quyết định quan trọng như fail-closed, deterministic rollout, RBAC và auditability khi hệ thống được refactor hoặc mở rộng.

4.2 Hạn chế và hướng phát triển

4.2.1 Hạn chế

Mặc dù đáp ứng mục tiêu mini project, hệ thống vẫn còn một số hạn chế khi so với nền tảng Feature Flag ở mức production:

- Hệ thống chưa được đánh giá dưới tải lớn; các kiểm thử hiện tập trung chủ yếu vào tính đúng đắn, luồng API và kịch bản demo.
- Kiến trúc hiện phù hợp với một backend service trong môi trường demo, chưa có cơ chế đồng bộ cấu hình/cache giữa nhiều instance.
- SDK mới có phiên bản JavaScript và vẫn gọi Evaluation API theo request/response, chưa hỗ trợ streaming hoặc push update thời gian thực.
- Demo backend hiện chỉ là minh họa server-side feature gating đơn giản, chưa phải Backend SDK hoàn chỉnh và chưa có middleware route guard tái sử dụng cho nhiều framework.
- Rule engine mới hỗ trợ các rule cơ bản như user allowlist, role targeting và percentage rollout, chưa có targeting theo thuộc tính phức tạp, thời gian, vị trí hoặc organization.
- Hệ thống chưa hỗ trợ multi-variant experiment và A/B testing hoàn chỉnh, dù response đã có khái niệm variant đơn giản.
- RBAC hiện phục vụ demo, chưa tích hợp OAuth2/OIDC, SSO, quản lý người dùng thật hoặc chính sách bảo mật ở mức doanh nghiệp.
- Chưa có approval workflow, stale flag detection, rate limiting đầy đủ, monitoring hạ tầng, backup/recovery và disaster recovery.
- Hệ thống mới được triển khai và đánh giá trên kiến trúc modular monolith, chưa xem xét đầy đủ các yêu cầu về hiệu năng, tính sẵn sàng và khả năng mở rộng trong môi trường microservices.

4.2.2 Hướng phát triển

Các hướng phát triển tương ứng với các hạn chế trên gồm:

- Bổ sung benchmark và load test cho Evaluation API, đo latency, throughput, cache hit/miss và hành vi khi database hoặc cache gặp lỗi.
- Mở rộng kiến trúc sang nhiều backend instance, dùng distributed cache hoặc event-based invalidation để đồng bộ snapshot cấu hình giữa các instance.
- Phát triển thêm SDK cho các ngôn ngữ phổ biến như Java, Go, Python hoặc C#, đồng thời nghiên cứu polling tối ưu, SSE/WebSocket hoặc streaming update.

- Phát triển Backend SDK hoặc middleware chính thức cho Node.js, NestJS hoặc Express để backend nghiệp vụ có thể guard route/service bằng Feature Flag một cách chuẩn hóa.
- Mở rộng Rule Evaluation Engine để hỗ trợ targeting theo thuộc tính người dùng, organization/tenant, thời gian, khu vực địa lý và điều kiện kết hợp.
- Thiết kế mô hình multi-variant flag và A/B testing, kết hợp statistics để đo lường hiệu quả từng biến thể.
- Thay thế mô hình RBAC demo bằng cơ chế xác thực chuẩn như OAuth2/OIDC hoặc SSO, kèm quản lý người dùng, nhóm quyền và chính sách truy cập chi tiết.
- Bổ sung approval workflow cho thay đổi production, stale flag detection để giảm feature flag debt, rate limiting, monitoring/alerting, backup/recovery và tài liệu vận hành production.
- Trong tương lai, hệ thống có thể được mở rộng để hỗ trợ kiến trúc microservices thông qua SDK cho backend, cache phân tán và cơ chế đồng bộ cấu hình theo thời gian thực giữa các service.

Tài liệu tham khảo

[1] Pete Hodgson. *Feature Toggles (aka Feature Flags)*. Martin Fowler. 9 **october** 2017. URL: https://martinfowler.com/articles/feature-toggles.html.

[2] Unleash. *Feature Flags*. Unleash Documentation. URL: https://docs.getunleash.io/concepts/feature-flags.

[3] LaunchDarkly. *Feature Flags*. LaunchDarkly Documentation. URL: https://docs.launchdarkly.com/home/flags.

[4] Microsoft. *How Microsoft Develops with DevOps*. Microsoft Learn. 28 **november** 2022. URL: https://learn.microsoft.com/en-us/devops/develop/how-microsoft-develops-devops.

[5] Unleash. *11 Best Practices for Building and Scaling Feature Flag Systems*. Unleash Documentation. URL: https://docs.getunleash.io/topics/feature-flags/feature-flag-best-practices.

[6] LaunchDarkly. *Targeting Rules*. LaunchDarkly Documentation. URL: https://launchdarkly.com/docs/home/flags/target-rules.

[7] LaunchDarkly. *Percentage Rollouts*. LaunchDarkly Documentation. URL: https://launchdarkly.com/docs/home/releases/percentage-rollouts.

[8] Rezvan Mahdavi-Hezaveh, Jacob Dremann **and** Laurie Williams. **?**Software Development with Feature Toggles: Practices Used by Practitioners**? in***Empirical Software Engineering*: 26, 1 (8 **january** 2021). DOI: 10.1007/s10664-020-09901-z. URL: https://link.springer.com/article/10.1007/s10664-020-09901-z.

[9] OWASP Foundation. *REST Security Cheat Sheet*. OWASP Cheat Sheet Series. URL: https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html.

[10] OWASP Foundation. *Logging Cheat Sheet*. OWASP Cheat Sheet Series. URL: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html.

[11] Docker. *Docker Compose*. Docker Documentation. URL: https://docs.docker.com/compose/.

[12] OpenFeature. *Flag Evaluation API*. OpenFeature Specification. URL: https://openfeature.dev/specification/sections/flag-evaluation/.

[13] PostgreSQL Global Development Group. *JSON Types*. PostgreSQL Documentation. URL: https://www.postgresql.org/docs/current/datatype-json.html.